
超经典 MySQL 练习 50 题

出品：Python 数据之道



24 Nov, 2020

Contents

1	超经典 MySQL 练习 50 题	4
1.1	建表和插入数据	4
1.2	题目 1	6
1.3	题目 2	8
1.4	题目 3	9
1.5	题目 4	11
1.6	题目 5	15
1.7	题目 6	16
1.8	题目 7	17
1.9	题目 8	19
1.10	题目 9	20
1.11	题目 10	24
1.12	题目 11	29
1.13	题目 12	31
1.14	题目 13	34
1.15	题目 14	37
1.16	题目 15	39
1.17	题目 16	42
1.18	题目 17	45
1.19	题目 18	49
1.20	题目 19	50
1.21	题目 20	55
1.22	题目 21	60
1.23	题目 22	61
1.24	题目 23	67
1.25	题目 24	74
1.26	题目 25	78
1.27	题目 26	84
1.28	题目 27	85
1.29	题目 28	86
1.30	题目 29	88
1.31	题目 30	89
1.32	题目 31	91

1.33 题目 32	91
1.34 题目 33	93
1.35 题目 34	94
1.36 题目 35	95
1.37 题目 36	96
1.38 题目 37	98
1.39 题目 38	99
1.40 题目 39	100
1.41 题目 40	101
1.42 题目 41	105
1.43 题目 42	108
1.44 题目 43	112
1.45 题目 44	113
1.46 题目 45	114
1.47 题目 46	116
1.48 题目 47	119
1.49 题目 48	120
1.50 题目 49	121
1.51 题目 50	122
2 延伸阅读	124
2.1 我是谁	124
2.2 推荐内容	125

1 超经典 MySQL 练习 50 题

出品：Python 数据之道 (ID:PyDataLab)

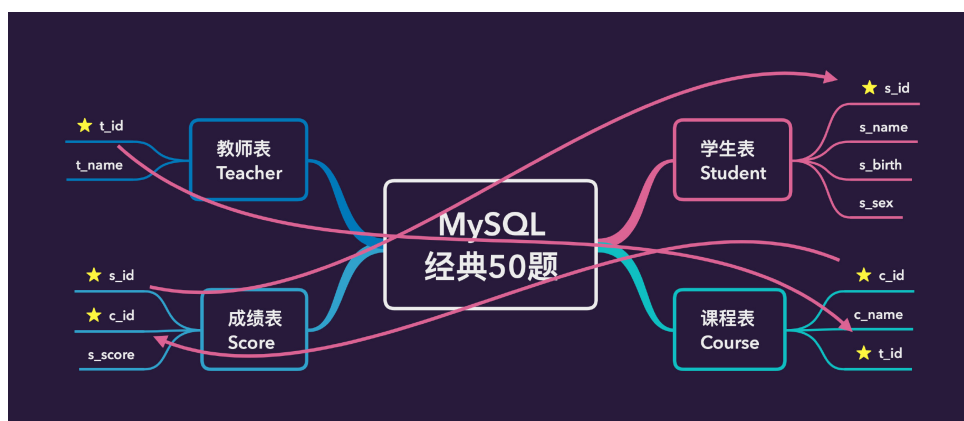
练习题整理：Peter

内容编辑：Lemon

相信大多学习了 Mysql 数据库语言的同学都会上网找练习来练手，而大部分的人肯定知道有一篇 Mysql 经典练习题 50 题的帖子，上面的题目基本上涵盖了 Mysql 查询语句的关键知识点。

笔者近期对又将这 50 题进行了练习，同时整理了相关的参考答案，有自己的思路和方法，也有参考大神们的。不得不说，这 50 题对 SQL 的提升真的很有帮助！

笔者使用的 MySQL 版本是 [MySQL 5.7.28](#)。



1.1 建表和插入数据

在开始之前，先建立本文所需要的数据表格：

```
1  -- 建表
2  -- 学生表
3  CREATE TABLE `Student` (
4      `s_id` VARCHAR(20),
5      `s_name` VARCHAR(20) NOT NULL DEFAULT '',
6      `s_birth` VARCHAR(20) NOT NULL DEFAULT '',
7      `s_sex` VARCHAR(10) NOT NULL DEFAULT '',
8      PRIMARY KEY(`s_id`)
9  );
```



```
10 -- 课程表
11 CREATE TABLE `Course` (
12     `c_id` VARCHAR(20),
13     `c_name` VARCHAR(20) NOT NULL DEFAULT '',
14     `t_id` VARCHAR(20) NOT NULL,
15     PRIMARY KEY(`c_id`)
16 );
17 -- 教师表
18 CREATE TABLE `Teacher` (
19     `t_id` VARCHAR(20),
20     `t_name` VARCHAR(20) NOT NULL DEFAULT '',
21     PRIMARY KEY(`t_id`)
22 );
23 -- 成绩表
24 CREATE TABLE `Score` (
25     `s_id` VARCHAR(20),
26     `c_id` VARCHAR(20),
27     `s_score` INT(3),
28     PRIMARY KEY(`s_id`,`c_id`)
29 );
30
31 -- 插入学生表测试数据
32 insert into Student values('01' , '赵雷' , '1990-01-01' , '男');
33 insert into Student values('02' , '钱电' , '1990-12-21' , '男');
34 insert into Student values('03' , '孙风' , '1990-05-20' , '男');
35 insert into Student values('04' , '李云' , '1990-08-06' , '男');
36 insert into Student values('05' , '周梅' , '1991-12-01' , '女');
37 insert into Student values('06' , '吴兰' , '1992-03-01' , '女');
38 insert into Student values('07' , '郑竹' , '1989-07-01' , '女');
39 insert into Student values('08' , '王菊' , '1990-01-20' , '女');
40
41 -- 课程表测试数据
42 insert into Course values('01' , '语文' , '02');
43 insert into Course values('02' , '数学' , '01');
44 insert into Course values('03' , '英语' , '03');
45
46 -- 教师表测试数据
47 insert into Teacher values('01' , '张三');
48 insert into Teacher values('02' , '李四');
49 insert into Teacher values('03' , '王五');
50
51 -- 成绩表测试数据
52 insert into Score values('01' , '01' , 80);
53 insert into Score values('01' , '02' , 90);
54 insert into Score values('01' , '03' , 99);
55 insert into Score values('02' , '01' , 70);
56 insert into Score values('02' , '02' , 60);
57 insert into Score values('02' , '03' , 80);
58 insert into Score values('03' , '01' , 80);
59 insert into Score values('03' , '02' , 80);
60 insert into Score values('03' , '03' , 80);
61 insert into Score values('04' , '01' , 50);
```

```
61 insert into Score values('04' , '02' , 30);
62 insert into Score values('04' , '03' , 20);
63 insert into Score values('05' , '01' , 76);
64 insert into Score values('05' , '02' , 87);
65 insert into Score values('06' , '01' , 31);
66 insert into Score values('06' , '03' , 34);
67 insert into Score values('07' , '02' , 89);
68 insert into Score values('07' , '03' , 98);
```

1.2 题目 1

1.2.1 题目要求

查询“01”课程比“02”课程成绩高的学生的信息及课程分数

1.2.2 SQL 实现

```
1  -- 方法1
2  select
3      a.*
4      ,b.s_score as 1_score
5      ,c.s_score as 2_score
6  from Student a
7  join Score b on a.s_id = b.s_id and b.c_id = '01'    -- 两个表通过学号连接，指
   定01
8  left join Score c on a.s_id = c.s_id and c.c_id='02' or c.c_id is NULL -- 指定
   02，或者c中的c_id直接不存在
9  -- 为NULL的条件可以不存在，因为左连接中会直接排除c表中不存在的数据，包含NULL
10 where b.s_score > c.s_score;    -- 判断条件
11
12
13 -- 方法2：直接使用where语句
14 select
15     a.*
16     ,b.s_score as 1_score
17     ,c.s_score as 2_score
18 from Student a, Score b, Score c
19 where a.s_id=b.s_id    -- 列出全部的条件
20 and a.s_id=c.s_id
21 and b.c_id='01'
22 and c.c_id='02'
23 and b.s_score > c.s_score;    -- 前者成绩高
```

```
mysql> select
-> a.*
-> ,b.s_score as 1_score
-> ,c.s_score as 2_score
-> from Student a
-> join Score b on a.s_id = b.s_id and b.c_id = '01' -- 两个表
通过学号连接, 指定01
-> left join Score c on a.s_id = c.s_id and c.c_id='02' or c.c_id
= NULL -- 指定02, 或者c中的c_id直接不存在
-> -- 为NULL的条件可以不存在, 因为左连接中会直接排除c表中不存在的数据, 包含NULL
[ -> where b.s_score > c.s_score; -- 判断条件
+-----+-----+-----+-----+-----+-----+
| s_id | s_name | s_birth | s_sex | 1_score | 2_score |
+-----+-----+-----+-----+-----+-----+
| 02 | 钱电 | 1990-12-21 | 男 | 70 | 60 |
| 04 | 李云 | 1990-08-06 | 男 | 50 | 40 |
+-----+-----+-----+-----+-----+-----+
2 rows in set (0.00 sec)
```

第二种方法实现:

```
mysql> select
-> a.*
-> ,b.s_score as 1_score
-> ,c.s_score as 2_score
-> from Student a, Score b, Score c
-> where a.s_id=b.s_id -- 列出全部的条件
-> and a.s_id=c.s_id
-> and b.c_id='01'
-> and c.c_id='02'
[ -> and b.s_score > c.s_score; -- 前者成绩高
+-----+-----+-----+-----+-----+-----+
| s_id | s_name | s_birth | s_sex | 1_score | 2_score |
+-----+-----+-----+-----+-----+-----+
| 02 | 钱电 | 1990-12-21 | 男 | 70 | 60 |
| 04 | 李云 | 1990-08-06 | 男 | 50 | 40 |
+-----+-----+-----+-----+-----+-----+
2 rows in set (0.01 sec)
```

方法2

```
[mysql> select * from Score;
+-----+-----+-----+
| s_id | c_id | s_score |
+-----+-----+-----+
| 01   | 01   | 80      |
| 01   | 02   | 90      |
| 01   | 03   | 96      |
| 02   | 01   | 70      |
| 02   | 02   | 60      |
| 02   | 03   | 80      |
| 03   | 01   | 80      |
| 03   | 02   | 81      |
| 03   | 03   | 85      |
| 04   | 01   | 50      |
| 04   | 02   | 40      |
| 04   | 03   | 30      |
| 05   | 01   | 76      |
| 05   | 02   | 87      |
| 06   | 01   | 43      |
| 06   | 03   | 56      |
| 07   | 02   | 89      |
| 07   | 03   | 94      |
+-----+-----+-----+
```

结果显示

1.3 题目 2

1.3.1 题目要求

查询“01”课程比“02”课程成绩低的信息及课程分数（题目 1 是成绩高）

1.3.2 SQL 实现

类比题目 1 的实现过程

```
1  -- 方法1：通过连接方式实现
2  select
3      a.*
4      ,b.s_score as 1_score
5      ,c.s_score as 2_score
6  from Student a
7  left join Score b on a.s_id=b.s_id and b.c_id='01' or b.c_id=NULL  -- 包含NULL
   的数据
8  join score c on a.s_id=c.s_id and c.c_id='02'
9  where b.s_score < c.s_score;
10
11 -- 通过where子句实现
12 select
13     a.*
14     ,b.s_score as 1_score
15     ,c.s_score as 2_score
16  from Student a, Score b, Score c
17  where a.s_id=b.s_id
18  and a.s_id=c.s_id
19  and b.c_id='01'
20  and c.c_id='02'
21  and b.s_score < c.s_score;  -- 前者比较小
```

1.4 题目 3

1.4.1 题目需求

查询平均成绩大于等于 60 分的同学的学生编号和学生姓名和平均成绩

1.4.2 SQL 实现

```
1  -- 执行顺序：先执行分组，再执行avg平均操作
2  select
3      b.s_id
4      ,b.s_name
5      ,round(avg(a.s_score), 2) as avg_score
6  from Student b
7  join Score a
8  on b.s_id = a.s_id
9  group by b.s_id  -- 分组之后查询每个人的平均成绩
10 having avg_score >= 60;
11
12 -- 附加题：总分超过200分的同学
13 select
```

```

14     b.s_id
15     ,b.s_name
16     ,round(sum(a.s_score),2) as sum_score    -- sum 求 和
17 from Student b
18 join Score a
19 on b.s_id=a.s_id
20 group by b.s_id
21 having sum_score > 200;

```

```

mysql> select b.s_id,b.s_name,round(avg(a.s_score),2) as avg_score
-> from Student b
-> join Score a
-> on b.s_id=a.s_id
-> group by b.s_id
-> having avg_score > 60
+-----+-----+-----+
| s_id | s_name | avg_score |
+-----+-----+-----+
| 01   | 赵雷   | 88.67    |
| 02   | 钱电   | 70.00    |
| 03   | 孙风   | 82.00    |
| 05   | 周梅   | 81.50    |
| 07   | 郑竹   | 91.50    |
+-----+-----+-----+

```

附加题：总分超过 200 分的同学

```

mysql> select
-> b.s_id
-> ,b.s_name
-> ,round(sum(a.s_score),2) as sum_score    -- sum求 和
-> from Student b
-> join Score a
-> on b.s_id=a.s_id
-> group by b.s_id
-> having sum_score > 200;
+-----+-----+-----+
| s_id | s_name | sum_score |
+-----+-----+-----+
| 01   | 赵雷   | 266.00    |
| 02   | 钱电   | 210.00    |
| 03   | 孙风   | 246.00    |
+-----+-----+-----+
3 rows in set (0.00 sec)

```

1.5 题目 4

1.5.1 题目要求

查询平均成绩小于 60 分的同学的学生编号和学生姓名和平均成绩（包括有成绩的和无成绩的）

1.5.2 SQL 实现 1-两种情况连接

平均分小于 60

```
1 select
2     b.s_id
3     ,b.s_name
4     ,round(avg(a.s_score), 2) as avg_score  -- round 四舍五入函数
5 from Student b
6 join Score a
7 on b.s_id = a.s_id
8 group by b.s_id  -- 分组之后查询每个人的平均成绩
9 having avg_score < 60;
```

结果为:

```
mysql> select
-> b.s_id
-> ,b.s_name
-> ,round(avg(a.s_score), 2) as avg_score  -- round四舍五入函数
-> from Student b
-> join Score a
-> on b.s_id = a.s_id
-> group by b.s_id  -- 分组之后查询每个人的平均成绩
-> having avg_score < 60;
+-----+-----+-----+
| s_id | s_name | avg_score |
+-----+-----+-----+
| 04   | 李云   | 40.00    |
| 06   | 吴兰   | 49.50    |
+-----+-----+-----+
2 rows in set (0.00 sec)
```

没有成绩的同学:

```
1 select
2     a.s_id
3     ,a.s_name
4     ,0 as avg_score
5 from Student a
6 where a.s_id not in ( -- 学生的学号不在给定表的学号中
7     select distinct s_id -- 查询出全部的学号
8     from Score
9 );
```

```
[mysql> select * from Student;
+-----+-----+-----+-----+
| s_id | s_name | s_birth | s_sex |
+-----+-----+-----+-----+
| 01   | 赵雷   | 1990-01-01 | 男     |
| 02   | 钱电   | 1990-12-21 | 男     |
| 03   | 孙风   | 1990-05-20 | 男     |
| 04   | 李云   | 1990-08-06 | 男     |
| 05   | 周梅   | 1991-12-01 | 女     |
| 06   | 吴兰   | 1992-03-01 | 女     |
| 07   | 郑竹   | 1989-07-02 | 女     |
| 08   | 王菊   | 1990-01-20 | 女     |
+-----+-----+-----+-----+
8 rows in set (0.00 sec)
```



```
[mysql> select * from Score;
```

s_id	c_id	s_score
01	01	80
01	02	90
01	03	96
02	01	70
02	02	60
02	03	80
03	01	80
03	02	81
03	03	85
04	01	50
04	02	40
04	03	30
05	01	76
05	02	87
06	01	43
06	03	56
07	02	89
07	03	94

18 rows in set (0.00 sec)

成绩表中没有08, 王菊

最后将两个部分的结果连起来即可：通过union方法

```
mysql> select
-> b.s_id
-> ,b.s_name
-> ,round(avg(a.s_score), 2) as avg_score -- round四舍五入函数
-> from Student b
-> join Score a
-> on b.s_id = a.s_id
-> group by b.s_id -- 分组之后查询每个人的平均成绩
-> having avg score < 60
-> union
-> select
-> a.s_id
-> ,a.s_name
-> ,0 as avg_score
-> from Student a
-> where a.s_id not in ( -- 学生的学号不在给定表的学号中
-> select distinct s_id -- 查询出全部的学号
-> from Score
-> );
```

s_id	s_name	avg_score
04	李云	40.00
06	吴兰	49.50
08	王菊	0.00

3 rows in set (0.01 sec)

1.5.3 SQL 实现 2-ifnull 函数判断

使用 ifnull 函数

```
1 select
2     S.s_id
3     ,S.s_name
4     ,round(avg(ifnull(C.s_score,0)), 2) as avg_score -- ifnull 函数：第一个参
      数存在则取它本身，不存在取第二个值0
5 from Student S
6 left join Score C
7 on S.s_id = C.s_id
8 group by s_id
9 having avg_score < 60;
```

使用 null 判断

```
1 select
2     a.s_id
3     ,a.s_name
4     ,ROUND(AVG(b.s_score), 2) as avg_score
5 from Student a
```

```
6 left join Score b on a.s_id = b.s_id
7 GROUP BY a.s_id
8 HAVING avg_score < 60 or avg_score is null; -- 最后的 NULL 判断
```

1.6 题目 5

1.6.1 题目需求

查询所有同学的学生编号、学生姓名、选课总数、所有课程的总成绩

1.6.2 SQL 实现

```
1 select
2     a.s_id
3     ,a.s_name
4     ,count(b.c_id) as course_number -- 课程个数
5     ,sum(b.s_score) as scores_sum -- 成绩总和
6 from Student a
7 left join Score b
8 on a.s_id = b.s_id
9 group by a.s_id,a.s_name;
```

```
mysql> select
-> a.s_id
-> ,a.s_name
-> ,count(b.c_id) as course_number
-> ,sum(b.s_score) as scores_sum
-> from Student a
-> left join Score b
-> on a.s_id = b.s_id
-> group by a.s_id,a.s_name;
+-----+-----+-----+-----+
| s_id | s_name | course_number | scores_sum |
+-----+-----+-----+-----+
| 01  | 赵雷   | 3             | 266        |
| 02  | 钱电   | 3             | 210        |
| 03  | 孙风   | 3             | 246        |
| 04  | 李云   | 3             | 120        |
| 05  | 周梅   | 2             | 163        |
| 06  | 吴兰   | 2             | 99         |
| 07  | 郑竹   | 2             | 183        |
| 08  | 王菊   | 0             | NULL       |
+-----+-----+-----+-----+
8 rows in set (0.00 sec)
```

1.7 题目 6

1.7.1 题目需求

查询“李”姓老师的数量

1.7.2 SQL 实现

```
1 select count(t_name) from Teacher where t_name like "李%"; -- 通配符
```

```
mysql> select * from Teacher;
+-----+-----+
| t_id | t_name |
+-----+-----+
| 01  | 张三  |
| 02  | 李四  |
| 03  | 王五  |
+-----+-----+
3 rows in set (0.01 sec)

mysql> select count(t_name) from Teacher where t_name like "李%";
+-----+
| count(t_name) |
+-----+
| 1 |
+-----+
1 row in set (0.01 sec)
```

这题怕是最简单的吧 ☹

1.8 题目 7

1.8.1 题目需求

查询学过张三老师授课的同学的信息

1.8.2 SQL 实现

```
1  -- 方法1：通过张三老师的课程的学生来查找；自己的方法
2  select * -- 3. 通过学号找出全部学生信息
3  from Student
4  where s_id in (
5      select s_id      -- 2.通过课程找出对应的学号
6      from Score S
7      join Course C
8      on S.c_id = C.c_id -- 课程表和成绩表
9      where C.t_id=(select t_id from Teacher where t_name="张三") -- 1.查询张三老
      师的课程
10 );
11
12 -- 方法2：通过张三老师的课程来查询
13 select s1.*
14 from Student s1
15 join Score s2
16 on s1.s_id=s2.s_id
17 where s2.c_id in (
18     select c_id from Course c where t_id=( -- 1. 通过老师找出其对应的课程
```

```

19     select t_id from Teacher t where t_name="张三"
20 )
21 )
22
23 -- 方法3
24 select s.* from Teacher t
25 left join Course c on t.t_id=c.t_id -- 教师表和课程表
26 left join Score sc on c.c_id=sc.c_id -- 课程表和成绩表
27 left join Student s on s.s_id=sc.s_id -- 成绩表和学生信息表
28 where t.t_name='张三';

```

自己的方法：

```

mysql> select * -- 3. 通过学号找出全部学生信息
-> from Student
-> where s_id in (
->   select s_id -- 2.通过课程找出对应的学号
->   from Score S
->   join Course C
->   on S.c_id = C.c_id
->   where C.t_id=(select t_id from Teacher where t_name="张三") -- 1.查询 张三老师的课程
-> );
+-----+-----+-----+-----+
| s_id | s_name | s_birth | s_sex |
+-----+-----+-----+-----+
| 01 | 赵雷 | 1990-01-01 | 男 |
| 02 | 钱电 | 1990-12-21 | 男 |
| 03 | 孙风 | 1990-05-20 | 男 |
| 04 | 李云 | 1990-08-06 | 男 |
| 05 | 周梅 | 1991-12-01 | 女 |
| 07 | 郑竹 | 1989-07-02 | 女 |
+-----+-----+-----+-----+
6 rows in set (0.00 sec)

```

方法 2 来实现：

```

mysql> -- 方法2
mysql> select s1.*
-> from Student s1
-> join Score s2
-> on s1.s_id=s2.s_id
-> where s2.c_id in (
->   select c_id from Course c where t_id=( -- 1. 通过老师找出其对应的课程
->     select t_id from Teacher t where t_name="张三"
->   )
-> );
+-----+-----+-----+-----+
| s_id | s_name | s_birth | s_sex |
+-----+-----+-----+-----+
| 01 | 赵雷 | 1990-01-01 | 男 |
| 02 | 钱电 | 1990-12-21 | 男 |
| 03 | 孙风 | 1990-05-20 | 男 |
| 04 | 李云 | 1990-08-06 | 男 |
| 05 | 周梅 | 1991-12-01 | 女 |
| 07 | 郑竹 | 1989-07-02 | 女 |
+-----+-----+-----+-----+
6 rows in set (0.00 sec)

```

方法 3 实现：

```
mysql> -- 方法3
mysql> select s.* from teacher t
      -> left join Course c on t.t_id=c.t_id
      -> left join Score sc on c.c_id=sc.c_id
      -> left join Student s on s.s_id=sc.s_id
      -> where t.t_name='张三';

+-----+-----+-----+-----+
| s_id | s_name | s_birth | s_sex |
+-----+-----+-----+-----+
| 01   | 赵雷   | 1990-01-01 | 男   |
| 02   | 钱电   | 1990-12-21 | 男   |
| 03   | 孙风   | 1990-05-20 | 男   |
| 04   | 李云   | 1990-08-06 | 男   |
| 05   | 周梅   | 1991-12-01 | 女   |
| 07   | 郑竹   | 1989-07-02 | 女   |
+-----+-----+-----+-----+
6 rows in set (0.01 sec)
```

1.9 题目 8

1.9.1 题目需求

找出没有学过张三老师课程的学生

1.9.2 SQL 实现

```
1 select * -- 3. 通过学号找出全部学生信息
2 from Student
3 where s_id not in ( -- 2.通过学号取反：学号不在张三老师授课的学生的学号中
4     select s_id
5     from Score S
6     join Course C
7     on S.c_id = C.c_id
8     where C.t_id=(select t_id from Teacher where t_name ="张三") -- 1.查询张三老
9         师的课程
10 );
11 -- 方法2:
12 select *
13 from Student s1
14 where s1.s_id not in (
```

```

15     select s2.s_id from Student s2 join Score s3 on s2.s_id=s3.s_id where s3.c_id
16         in(
17         select c.c_id from Course c join Teacher t on c.t_id=t.t_id where t_name="
18             张三"
19     )
20 );
21
22 -- 方法 3
23 select s1.*
24 from Student s1
25 join Score s2
26 on s1.s_id=s2.s_id
27 where s2.c_id not in (
28     select c_id from Course c where t_id=( -- 1. 通过老师找出其对应的课程
29         select t_id from Teacher t where t_name="张三"
30     )
31 );

```

```

mysql> select * -- 3. 通过学号找出全部学生信息
-> from Student
-> where s_id not in ( -- 2.通过学号取反：学号不在张三老师授课的学生的学号中
->     select s_id
->     from Score S
->     join Course C
->     on S.c_id = C.c_id
->     where C.t_id=(select t_id from Teacher where t_name="张三") -- 1.查询张三老师的课程
-> );
+-----+-----+-----+-----+
| s_id | s_name | s_birth | s_sex |
+-----+-----+-----+-----+
| 06   | 吴兰   | 1992-03-01 | 女   |
| 08   | 王菊   | 1990-01-20 | 女   |
+-----+-----+-----+-----+
2 rows in set (0.00 sec)

```

方法 2:

```

mysql> select *
-> from Student s1
-> where s1.s_id not in (
->     select s2.s_id from Student s2 join Score s3 on s2.s_id=s3.s_id where s3.c_id in(
->         select c.c_id from Course c join Teacher t on c.t_id=t.t_id where t_name="张三"
->     )
-> );
+-----+-----+-----+-----+
| s_id | s_name | s_birth | s_sex |
+-----+-----+-----+-----+
| 06   | 吴兰   | 1992-03-01 | 女   |
| 08   | 王菊   | 1990-01-20 | 女   |
+-----+-----+-----+-----+
2 rows in set (0.01 sec)

```

1.10 题目 9

1.10.1 题目需求

查询学过编号为 01，并且学过编号为 02 课程的学生信息

1.10.2 SQL 实现

```
1  -- 自己的方法：通过自连接实现
2  select s1.*
3  from Student s1
4  where s_id in (
5      select s2.s_id from Score s2
6      join Score s3
7      on s2.s_id=s3.s_id
8      where s2.c_id='01' and s3.c_id='02'
9  );
10
11 -- 方法2：直接通过where语句实现
12 select s1.*
13 from Student s1, Score s2, Score s3
14 where s1.s_id=s2.s_id
15 and s1.s_id=s3.s_id
16 and s2.c_id=01 and s3.c_id=02;
17
18 -- 方法3：两个子查询
19 -- 1. 先查出学号
20 select sc1.s_id
21 from (select * from Score s1 where s1.c_id='01') sc1,
22      (select * from Score s1 where s1.c_id='02') sc2
23 where sc1.s_id=sc2.s_id;
24
25 -- 2. 找出学生信息
26 select *
27 from Student
28 where s_id in (select sc1.s_id -- 指定学号是符合要求的
29               from (select * from Score s1 where s1.c_id='01') sc1,
30                    (select * from Score s1 where s1.c_id='02') sc2
31               where sc1.s_id=sc2.s_id);
```

1. 先从 Score 表中看看哪些人是满足要求的：**01-05** 同学是满足的

```
[mysql> select * from Score;
+-----+-----+-----+
| s_id | c_id | s_score |
+-----+-----+-----+
| 01   | 01   | 80      |
| 01   | 02   | 90      |
| 01   | 03   | 96      |
| 02   | 01   | 70      |
| 02   | 02   | 60      |
| 02   | 03   | 80      |
| 03   | 01   | 80      |
| 03   | 02   | 81      |
| 03   | 03   | 85      |
| 04   | 01   | 50      |
| 04   | 02   | 40      |
| 04   | 03   | 30      |
| 05   | 01   | 76      |
| 05   | 02   | 87      |
| 06   | 01   | 43      |
| 06   | 03   | 56      |
| 07   | 02   | 89      |
| 07   | 03   | 94      |
+-----+-----+-----+
18 rows in set (0.00 sec)
```

通过自连接查询的语句如下：

```
mysql> select s2.s_id from Score s2
-> join Score s3
-> on s2.s_id=s3.s_id
[ -> where s2.c_id='01' and s3.c_id='02';
+-----+
| s_id |
+-----+
| 01   |
| 02   |
| 03   |
| 04   |
| 05   |
+-----+
5 rows in set (0.00 sec)
```

查询出学号后再匹配出学生信息：

```
mysql> select s1.*
-> from Student s1
-> where s_id in (
-> select s2.s_id from Score s2
-> join Score s3
-> on s2.s_id=s3.s_id
-> where s2.c_id='01' and s3.c_id='02'
[ -> );
```

第二步

```
+-----+-----+-----+-----+
| s_id | s_name | s_birth | s_sex |
+-----+-----+-----+-----+
| 01   | 赵雷   | 1990-01-01 | 男   |
| 02   | 钱电   | 1990-12-21 | 男   |
| 03   | 孙风   | 1990-05-20 | 男   |
| 04   | 李云   | 1990-08-06 | 男   |
| 05   | 周梅   | 1991-12-01 | 女   |
+-----+-----+-----+-----+
5 rows in set (0.00 sec)
```

第一步

通过 where 语句实现：

```
mysql> select s1.*
-> from Student s1, Score s2, Score s3
-> where s1.s_id=s2.s_id
-> and s1.s_id=s3.s_id
-> and s2.c_id=01 and s3.c_id=02; where语句实现
```

s_id	s_name	s_birth	s_sex
01	赵雷	1990-01-01	男
02	钱电	1990-12-21	男
03	孙风	1990-05-20	男
04	李云	1990-08-06	男
05	周梅	1991-12-01	女

5 rows in set (0.00 sec)

方法 3 的实现：

```
mysql> select *
-> from Student
-> where s_id in (select sc1.s_id -- 指定学号是符合要求的
-> from (select * from Score s1 where s1.c_id='01') sc1,
-> (select * from Score s1 where s1.c_id='02') sc2
-> where sc1.s_id=sc2.s_id);
```

s_id	s_name	s_birth	s_sex
01	赵雷	1990-01-01	男
02	钱电	1990-12-21	男
03	孙风	1990-05-20	男
04	李云	1990-08-06	男
05	周梅	1991-12-01	女

5 rows in set (0.00 sec)

1.11 题目 10

1.11.1 题目需求

查询学过 01 课程，但是没有学过 02 课程的学生信息（注意和上面 8 题目的区别）

1.11.2 SQL 实现

首先看看哪些同学是满足要求的：只有 06 号同学是满足的

```
[mysql> select * from Score;
+-----+-----+-----+
| s_id | c_id | s_score |
+-----+-----+-----+
| 01   | 01   | 80      |
| 01   | 02   | 90      |
| 01   | 03   | 96      |
| 02   | 01   | 70      |
| 02   | 02   | 60      |
| 02   | 03   | 80      |
| 03   | 01   | 80      |
| 03   | 02   | 81      |
| 03   | 03   | 85      |
| 04   | 01   | 50      |
| 04   | 02   | 40      |
| 04   | 03   | 30      |
| 05   | 01   | 76      |
| 05   | 02   | 87      |
| 06   | 01   | 43      |
| 06   | 03   | 56      |
| 07   | 02   | 89      |
| 07   | 03   | 94      |
+-----+-----+-----+
18 rows in set (0.00 sec)
```

1.11.2.1 错误思路 1

直接将上面一题的结果全部排出，导致那些没有学过 01 课程的学生也出现了：07，08

```

1 select s1.*
2 from Student s1
3 where s_id not in (    -- 直接将上面一题的结果全部排出，导致那些没有学过01课程的
                        学生也出现了：07，08
4     select s2.s_id from Score s2
5     join Score s3
6     on s2.s_id=s3.s_id
7     where s2.c_id='01' and s3.c_id='02'
8 );

```

```

mysql> select s1.*
      -> from Student s1
      -> where s_id not in (
      ->     select s2.s_id from Score s2
      ->     join Score s3
      ->     on s2.s_id=s3.s_id
      ->     where s2.c_id='01' and s3.c_id='02'
      -> );
+-----+-----+-----+-----+
| s_id | s_name | s_birth | s_sex |
+-----+-----+-----+-----+
| 06   | 吴兰   | 1992-03-01 | 女   |
| 07   | 郑竹   | 1989-07-02 | 女   |
| 08   | 王菊   | 1990-01-20 | 女   |
+-----+-----+-----+-----+
3 rows in set (0.00 sec)

```

1.11.2.2 错误思路 2

将上面题目中的 02 课程直接取反，导致同时修过 01，02，03 或者只修 01，03 的同学也会出现

```

1 select s1.*
2 from Student s1
3 where s_id in (
4     select s2.s_id from Score s2
5     join Score s3
6     on s2.s_id=s3.s_id
7     where s2.c_id='01' and s3.c_id != '02'    -- 直接取反是不行的，因为修改
                                                (01,02,03) 的同学也会出现
8 );

```

```
mysql> select s1.*
-> from Student s1
-> where s_id in (
->   select s2.s_id from Score s2
->   join Score s3
->   on s2.s_id=s3.s_id
->   where s2.c_id='01' and s3.c_id != '02' -- 直接取反是不行的，因为修改 (01,02,03)的同学也会出现
-> );
```

s_id	s_name	s_birth	s_sex
01	赵雷	1990-01-01	男
02	钱电	1990-12-21	男
03	孙风	1990-05-20	男
04	李云	1990-08-06	男
05	周梅	1991-12-01	女
06	吴兰	1992-03-01	女

6 rows in set (0.00 sec)

同时修3门课的同学也会出现

1.11.2.3 正确思路

<https://www.jianshu.com/p/9abffdd334fa>

```
1 -- 方法1：根据两种修课情况来判断
2
3 select s1.*
4 from Student s1
5 where s1.s_id in (select s_id from Score where c_id='01') -- 修过01课程，要保
   留
6 and s1.s_id not in (select s_id from Score where c_id='02'); -- 哪些人修过02，
   需要排除
```

```
mysql> select s1.*
-> from Student s1
-> where s1.s_id in (select s_id from Score where c_id='01')
-> and s1.s_id not in (select s_id from Score where c_id='02');
```

s_id	s_name	s_birth	s_sex
06	吴兰	1992-03-01	女

1 row in set (0.00 sec)

```
[mysql> select s_id from Score where c_id='01';
+-----+
| s_id |
+-----+
| 01   |
| 02   |
| 03   |
| 04   |
| 05   |
| 06   |
+-----+
6 rows in set (0.00 sec)
```

课程01出现的学号，需要保留

```
[mysql> select s_id from Score where c_id='02';
+-----+
| s_id |
+-----+
| 01   |
| 02   |
| 03   |
| 04   |
| 05   |
| 07   |
+-----+
6 rows in set (0.00 sec)
```

课程02出现的学号，需要排除

!!!! 方法2：先把06号学生找出来

```
1 select * from Student where s_id in (
2   select s_id
3   from Score
4   where c_id='01'    -- 修过01课程的学号
5   and s_id not in (select s_id    -- 同时学号不能在修过02课程中出现
6                     from Score
7                     where c_id='02')
8 );
```



```
mysql> select s_id
-> from Score
-> where c_id='01'
-> and s_id not in (select s_id from Score where c_id='02');
+-----+
| s_id |
+-----+
| 06   |
+-----+
1 row in set (0.00 sec)
```

```
mysql> select * from Student where s_id in (
->     select s_id
->     from Score
->     where c_id='01'
->     and s_id not in (select s_id
->         主体部分      from Score
->                       where c_id='02')
-> );
+-----+-----+-----+-----+
| s_id | s_name | s_birth | s_sex |
+-----+-----+-----+-----+
| 06   | 吴兰   | 1992-03-01 | 女    |
+-----+-----+-----+-----+
1 row in set (0.00 sec)
```

1.12 题目 11

1.12.1 题目需求

查询没有学完全部课程的同学的信息

1.12.2 SQL 实现

```
1 -- 自己的方法
2 select *    -- 排除学号后得到的结果
3 from Student
4 where s_id not in (select s_id from (select s_id, count(s_id) as number -- 3.
5                               最大课程数所在的学号需要排除
6                               from Score
```

```

6         group by s_id) s -- 取别名
7 where number=(select max(number) -- 2.保证最大的课程数
8         from( select s_id, count(s_id) as number -- 1.学号和个数统计 (即
           修了几门课)
9         from Score group by s_id)t)); -- 别名

```

```

mysql> select *
-> from Student
-> where s_id not in (select s_id from (select s_id, count(s_id) as number -- 3.最大课程数所在的学号
->         from Score
->         group by s_id) s
-> where number=(select max(number) -- 2.保证最大的课程数
->         from( select s_id, count(s_id) as number -- 1.学号和个数统计 (即修了几门课)
->         from Score group by s_id)t));
+-----+-----+-----+-----+
| s_id | s_name | s_birth | s_sex |
+-----+-----+-----+-----+
| 05   | 周梅   | 1991-12-01 | 女   |
| 06   | 吴兰   | 1992-03-01 | 女   |
| 07   | 郑竹   | 1989-07-02 | 女   |
| 08   | 王菊   | 1990-01-20 | 女   |
+-----+-----+-----+-----+
4 rows in set (0.00 sec)

```

自己的方法一开始在课程的最大数中没有使用 Course 表，导致多使用了一个临时表的结果，现在改成使用 Course 表的统计值 (3) 作为课程的总数：

```

1 select s.*
2 from Student s
3 where s_id not in (
4     select s_id
5     from Score s1
6     group by s_id
7     having count(*) = (select count(*) from Course)
8 );

```

```

mysql> select s.*
-> from Student s
-> where s_id not in (
->     select s_id
->     from Score s1
->     group by s_id
->     having count(*) = (select count(*) from Course)
-> );
+-----+-----+-----+-----+
| s_id | s_name | s_birth | s_sex |
+-----+-----+-----+-----+
| 05   | 周梅   | 1991-12-01 | 女   |
| 06   | 吴兰   | 1992-03-01 | 女   |
| 07   | 郑竹   | 1989-07-02 | 女   |
| 08   | 王菊   | 1990-01-20 | 女   |
+-----+-----+-----+-----+
4 rows in set (0.00 sec)

```

```
1 -- 方法2: having
2
3 select s.*
4 from Student s -- 学生表
5 left join Score s1 -- 成绩表
6 on s1.s_id = s.s_id
7 group by s.s_id -- 学号分组
8 having count(s1.c_id) < ( -- 分组后学生的课程数 < 3
9     select count(*) from Course -- 全部课程数 = 3
10 )
```

1.13 题目 12

1.13.1 题目需求

查询至少有一门课与学号为 01 的同学所学相同的同学的信息

1.13.2 SQL 实现

首先看看结果的：因为 01 号同学修了全部课程，所以其他的同学都是满足要求，除了 08 号同学没有任何成绩，不符合。

```
[mysql> select * from Score;
```

s_id	c_id	s_score
01	01	80
01	02	90
01	03	96
02	01	70
02	02	60
02	03	80
03	01	80
03	02	81
03	03	85
04	01	50
04	02	40
04	03	30
05	01	76
05	02	87
06	01	43
06	03	56
07	02	89
07	03	94

18 rows in set (0.00 sec)

具体实现过程为：

```
1 select *      -- 3、求出学生信息
2 from Student
3 where s_id in (
4     select distinct s_id  -- 2、找出满足课程在01学生课程中的全部学号（学生），学
        号去重，同时将01自己排除
5     from Score
6     where c_id in (
```

```

7      select c_id
8      from Score
9      where s_id=01    -- 1、找出学号01同学的全部课程
10     )
11    and s_id != 01);

```

```

mysql> select *    -- 3、求出学生信息
      -> from Student
      -> where s_id in (
      ->   select distinct s_id    -- 2、找出满足课程在01学生课程中的全部学号（学生），学号去重，同时将01自己排除
      ->   from Score
      ->   where c_id in (select c_id
      ->                     from Score
      ->                     where s_id=01    -- 1、找出学号01同学的全部课程
      ->                     )
      ->   and s_id != 01);
+-----+-----+-----+-----+
| s_id | s_name | s_birth | s_sex |
+-----+-----+-----+-----+
| 02   | 钱电   | 1990-12-21 | 男   |
| 03   | 孙风   | 1990-05-20 | 男   |
| 04   | 李云   | 1990-08-06 | 男   |
| 05   | 周梅   | 1991-12-01 | 女   |
| 06   | 吴兰   | 1992-03-01 | 女   |
| 07   | 郑竹   | 1989-07-02 | 女   |
+-----+-----+-----+-----+
6 rows in set (0.00 sec)

```

```

1  -- 方法2
2  select s1.*
3  from Student s1
4  join Score s2
5  on s1.s_id = s2.s_id    -- 学生表和成绩表的关联
6  and c_id in (select c_id from Score where s_id=01)    -- 对课程进行限制，只在01学
   生的课程内
7  group by s1.s_id;    -- 根据学号分组

```

```

mysql> select s1.*
      -> from Student s1
      -> join Score s2
      -> on s1.s_id = s2.s_id    -- 学生表和成绩表的关联
      -> and c_id in (select c_id from Score where s_id=01)    -- 课程在01所学的课程内
      -> group by s1.s_id;    -- 根据学号分组
+-----+-----+-----+-----+
| s_id | s_name | s_birth | s_sex |
+-----+-----+-----+-----+
| 01   | 赵雷   | 1990-01-01 | 男   |
| 02   | 钱电   | 1990-12-21 | 男   |
| 03   | 孙风   | 1990-05-20 | 男   |
| 04   | 李云   | 1990-08-06 | 男   |
| 05   | 周梅   | 1991-12-01 | 女   |
| 06   | 吴兰   | 1992-03-01 | 女   |
| 07   | 郑竹   | 1989-07-02 | 女   |
+-----+-----+-----+-----+
7 rows in set (0.00 sec)

```

1.14 题目 13

1.14.1 题目需求

查询和 01 同学学习的课程完全相同的同学的信息

1.14.2 SQL 实现

1. 自己的方法

```

1 select *
2 from Student
3 where s_id in (select s_id -- 3、步骤2中得到的学号是满足要求的
4                from(select distinct(s_id), count(c_id) number
5                      from Score
6                      group by s_id)t1 -- 1、学号和所修课程分组的结果t1
7                where number=3 -- 2、投机：选择出所修课程数是3（01修了3门）的学
                        号
8                and s_id !=01 -- 01 本身排除
9                );

```

```

mysql> select *
-> from Student
-> where s_id in (select s_id -- 3、步骤2中得到的学号是满足要求的
->                from(select distinct(s_id), count(c_id) number
->                      from Score
->                      group by s_id)t1 -- 1、学号和所修课程分组的结果t1
->                where number=3 -- 2、选择出所修课程数是3（01修了3门）的学号
->                and s_id !=01 -- 01 本身排除
->                );
+-----+-----+-----+-----+
| s_id | s_name | s_birth | s_sex |
+-----+-----+-----+-----+
| 02  | 钱电   | 1990-12-21 | 男    |
| 03  | 孙凤   | 1990-05-20 | 男    |
| 04  | 李云   | 1990-08-06 | 男    |
+-----+-----+-----+-----+
3 rows in set (0.01 sec)

```

修满3门课

我们在上面的步骤 2 中不考虑直接指定 3（where number=3），而是用 01 学生所修的课程数（虽然也是 3）来代替：

```

1 select *
2 from Student
3 where s_id in(
4     select s_id -- 3、步骤2中得到的学号是满足要求的
5     from(select distinct(s_id), count(c_id) number
6         from Score
7         group by s_id)t1 -- 1、学号和所修课程分组的结果t1
8     where number=(select count(c_id) number
9                   from Score
10                  group by s_id having s_id=01) -- 2、改变的地方：使用学号
                                                01的课程数3来代替

```

```

11     and s_id != 01 -- 01 本身排除
12 );

```

```

mysql> select *
-> from Student
-> where s_id in(
->     select s_id -- 3、步骤2中得到的学号是满足要求的
->     from(select distinct(s_id), count(c_id) number
->         from Score
->         group by s_id)t1 -- 1、学号和所修课程分组的结果t1
->     where number=(select count(c_id) number
->                   from Score
->                   group by s_id having s_id=01) -- 2、直接用01的课程数3来代替
->     and s_id != 01 -- 01 本身排除
-> );
+-----+-----+-----+-----+
| s_id | s_name | s_birth | s_sex |
+-----+-----+-----+-----+
| 02   | 钱电   | 1990-12-21 | 男   |
| 03   | 孙风   | 1990-05-20 | 男   |
| 04   | 李云   | 1990-08-06 | 男   |
+-----+-----+-----+-----+
3 rows in set (0.00 sec)

```

2. 使用group_concat函数

group_concat的使用方法为：

```

1 group_concat([DISTINCT] 字段 [Order BY ASC/DESC 排序字段] [Separator '分隔符'])

```

我们将Score表中每个s_id的c_id进行分组合并，实际的效果如下：

```

1 select
2     s_id
3     ,group_concat(c_id order by c_id) -- 分组合并，同时排序
4 from Score
5 group by s_id;

```

```
mysql> select
  -> s_id
  -> ,group_concat(c_id order by c_id) -- 分组合并，同时排序
  -> from Score
  -> group by s_id;
+-----+-----+
| s_id | group_concat(c_id order by c_id) |
+-----+-----+
| 01   | 01,02,03                        |
| 02   | 01,02,03                        |
| 03   | 01,02,03                        |
| 04   | 01,02,03                        |
| 05   | 01,02                          |
| 06   | 01,03                          |
| 07   | 02,03                          |
+-----+-----+
7 rows in set (0.00 sec)
```

需要进行排序的原因是防止出现这种情况：01 修的课程顺序是：01，02，03；如果有同学修课的顺序是 02，03，01，虽然顺序不同，但是本质上他们修的课程是相同的

使用排序后都会变成：**01，02，03**，保证结果相同

那么之后，我们只需要判断合并后和 01 号同学相同的结果即可，取出学号：

```
1 select *      -- 3、查询信息
2 from Student
3 where s_id in(
4   select s_id
5   from Score
6   group by s_id
7   having group_concat(c_id order by c_id)=( -- 2、找出和01号学生分组合并结果
      相同的学号s_id; 也需要排序
8     select group_concat(c_id order by c_id) -- 1、找出01号学生分组合并的结果，同时排序；排序很重要
9     from Score
10    group by s_id
11    having s_id=01)
12 and s_id != 01 -- 将自身排除
13 );
```



```
mysql> select *      -- 3、查询信息
-> from Student
-> where s_id in(
->   select s_id
->   from Score
->   group by s_id
->   having group_concat(c_id order by c_id)=(
->     select group_concat(c_id order by c_id)
->     from Score
->     group by s_id
->     having s_id=01)
-> and s_id != 01  -- 将自身排除
-> );
```

s_id	s_name	s_birth	s_sex
02	钱电	1990-12-21	男
03	孙风	1990-05-20	男
04	李云	1990-08-06	男

3 rows in set (0.00 sec)

1.15 题目 14

1.15.1 题目需求

查询没有修过张三老师讲授的任何一门课程的学生姓名

1.15.2 SQL 实现

自己的方法，具体过程如下：

```
1 select s_name      -- 4、学号取反找到学生姓名
2 from Student
3 where s_id not in(
4   select distinct(s_id) -- 3、课程号找到对应的学号
5   from Score
6   where c_id=(
7     select c_id -- 2、教师编号找到对应的课程号
8     from Course
9     where t_id=(
```

```
10     select t_id    -- 1、姓名找到教师编号
11     from Teacher
12     where t_name='张三')
13 );
```

```
mysql> select s_name    -- 4、学号取反找到学生姓名
-> from Student
-> where s_id not in(
->     select distinct(s_id) -- 3、课程号找到对应的学号
->     from Score
->     where c_id=(
->         select c_id -- 2、教师编号找到对应的课程号
->         from Course
->         where t_id=(
->             select t_id    -- 1、姓名找到教师编号
->             from Teacher
->             where t_name='张三')
->     );
+-----+
| s_name |
+-----+
| 吴兰   |
| 王菊   |
+-----+
2 rows in set (0.00 sec)
```

```
1  -- 修过张老师课程的学生的学号
2  select distinct(s_id)
3  from Score
4  where c_id=(
5      select c_id
6      from Course
7      where t_id=(
8          select t_id
9          from Teacher
10         where t_name='张三')); -- 修过张三老师课的学生
```

```
mysql> select distinct(s_id)
-> from Score
-> where c_id=(
->   select c_id
->   from Course
->   where t_id=(
->     select t_id
->     from Teacher
->     where t_name='张三'); -- 修过张三老师课的学生
+-----+
| s_id |
+-----+
| 01   |
| 02   |
| 03   |
| 04   |
| 05   |
| 07   |
+-----+
6 rows in set (0.00 sec)
```

1.16 题目 15

1.16.1 题目需求

查询两门及其以上不及格课程的同学的学号，姓名及其平均成绩

1.16.2 SQL 实现

首先看看哪些同学是满足两门或者两门以上是及格的

```
mysql> select s_id
-> from Score
-> where s_score >= 60
-> group by s_id
-> having count(s_score) >= 2; -- 2门课程大于等于60分的数量
+-----+
| s_id |
+-----+
| 01   |
| 02   |
| 03   |
| 05   |
| 07   |
+-----+
5 rows in set (0.00 sec)
```

```

1  -- 2门及以上不及格的
2
3  select
4      s_id
5      ,round(avg(s_score))  avg_score
6  from Score
7  where s_score < 60 -- 小于60分，不及格
8  group by s_id
9  having count(s_score) >= 2;    -- 不及格的有2门以上

```

说明 04, 06 是我们最终想要的结果

```

mysql> select
      -> s_id
      -> ,round(avg(s_score))  avg_score
      -> from Score
      -> where s_score < 60
      -> group by s_id
      -> having count(s_score) >= 2;    -- 2门课程大于等于60分的数量
+-----+-----+
| s_id | avg_score |
+-----+-----+
| 04   |         40 |
| 06   |         50 |
+-----+-----+

```

```

1  -- 自己的方法
2  select
3      s.s_id
4      ,s.s_name
5      ,t.avg_score
6  from Student s
7  join (select
8          s_id
9          ,round(avg(s_score))  avg_score
10         from Score
11         where s_score < 60
12         group by s_id
13         having count(s_score) >= 2)t
14  on s.s_id=t.s_id

```

```
mysql> select
-> s.s_id
-> ,s.s_name
-> ,t.avg_score
-> from Student s
-> join (select
->     s_id
->     ,round(avg(s_score)) avg_score
->     from Score
->     where s_score < 60
->     group by s_id
->     having count(s_score) >= 2)t
-> on s.s_id=t.s_id;
+-----+-----+-----+
| s_id | s_name | avg_score |
+-----+-----+-----+
| 04  | 李云  |         40 |
| 06  | 吴兰  |         50 |
+-----+-----+-----+
2 rows in set (0.00 sec)
```

```
1  -- 参考方法1
2
3  select
4      a.s_id,
5      a.s_name,
6      ROUND(AVG(b.s_score))
7  from Student a
8  left join Score b
9  on a.s_id = b.s_id
10 where a.s_id in(
11     select s_id
12     from Score
13     where s_score<60
14     group by s_id
15     having count(1)>=2)
16 group by a.s_id,a.s_name
17
18 -- 参考方法2
19 select
20     s.s_id
21     ,s_name
22     ,round(avg(s_score), 2) avg_score
23 from Student s
24 join Score sc
25 on s.s_id=sc.s_id
26 and sc.s_score < 60  -- 不及格
```

```
27 group by s.s_id    -- 学号分组
28 having count(sc.c_id )>= 2;    -- 2 门课
```

1.16.3 改进点

上面的两种方法都没有考虑都 08 学生，3 门都没有成绩，这个本题需要改进的地方。

1.17 题目 16

1.17.1 题目需求

检索 01 课程分数小于 60，按分数降序排列的学生信息

1.17.2 SQL 实现

自己的方法如下：

首先从 Score 表中找出哪些学生是满足这个要求：

```
1 select
2     s_id
3     ,s_score
4 from Score
5 where s_score < 60
6 and c_id = 01;
```

```
mysql> select
-> s_id
-> ,s_score
-> from Score
-> where s_score < 60
[
-> and c_id = 01;
+-----+-----+
| s_id | s_score |
+-----+-----+
| 04   | 50      |
| 06   | 43      |
+-----+-----+
2 rows in set (0.01 sec)
```



然后将上面的结果和 Student 表查询：

```
1 select s.*
2 from Student s
3 where s.s_id in (
4     select
5         s_id
6         ,s_score
7     from Score
8     where s_score < 60
9     and c_id = 01
10 );
```

```
mysql> select s.*
-> from Student s
-> where s.s_id in (
->   select
->   s_id
-> from Score
-> where s_score < 60
-> and c_id = 01
-> );
```

s_id	s_name	s_birth	s_sex
04	李云	1990-08-06	男
06	吴兰	1992-03-01	女

2 rows in set (0.00 sec)

```
1 select
2     s.*
3     ,t.s_score
4 from Student s
5 join (select s_id,s_score -- 2、Student和t的连接查询
6       from Score
7       where s_score < 60
8       and c_id = 01 )t -- 1、将第一步结果作为中间表t
9 on s.s_id=t.s_id;
```

```
mysql> select
-> s.*
-> ,t.s_score
-> from Student s
-> join (select s_id,s_score -- 2、Student和t的连接查询
->       from Score
->       where s_score < 60
->       and c_id = 01 )t -- 1、将第一步结果作为中间表t
-> on s.s_id=t.s_id;
```

s_id	s_name	s_birth	s_sex	s_score
04	李云	1990-08-06	男	50
06	吴兰	1992-03-01	女	43

2 rows in set (0.00 sec)


```
1 -- 自己的方法2：两个表的直接连接查询+where条件
2 select
3     s.*
4     ,sc.s_score
5 from Student s
6 join Score sc
7 on s.s_id=sc.s_id
8 where sc.c_id=01 and sc.s_score < 60
9 order by sc.s_score desc; -- 默认就是降序desc
```

```
mysql> select
-> s.*
-> ,sc.s_score
-> from Student s
-> join Score sc
-> on s.s_id=sc.s_id
-> where sc.c_id=01 and sc.s_score < 60
[ -> order by sc.s_score desc; -- 默认就是降序desc
+-----+-----+-----+-----+-----+
| s_id | s_name | s_birth   | s_sex | s_score |
+-----+-----+-----+-----+-----+
| 04   | 李云   | 1990-08-06 | 男    | 50      |
| 06   | 吴兰   | 1992-03-01 | 女    | 43      |
+-----+-----+-----+-----+-----+
2 rows in set (0.00 sec)
```

1.18 题目 17

1.18.1 题目需求

按平均成绩从高到低（降序）显示所有学生的所有课程的成绩以及平均成绩

1.18.2 SQL 实现

下面是自己的解法：

1、先求出每个同学的平均分，并降序排列

```
1 select
2     s_id
3     ,round(avg(s_score),2) avg_score
4 from Score
5 group by s_id
6 order by 2 desc;
```

```
mysql> select
-> s_id
-> ,round(avg(s_score),2) avg_score
-> from Score
-> group by s_id
-> order by 2 desc;
+-----+-----+
| s_id | avg_score |
+-----+-----+
| 07   | 91.50    |
| 01   | 88.67    |
| 03   | 82.00    |
| 05   | 81.50    |
| 02   | 70.00    |
| 06   | 49.50    |
| 04   | 40.00    |
+-----+-----+
7 rows in set (0.00 sec)
```

```
1 -- 自己的方法
2 select
3     s.s_id
4     ,s.c_id
5     ,s.s_score
6     ,t.avg_score
7 from Score s
8 join (select
9     s_id
10    ,round(avg(s_score),2) avg_score
11    from Score
12    group by s_id)t
13 on s.s_id = t.s_id
14 order by 4 desc; -- 指的是第4个字段
```

```
mysql> select
-> s.s_id
-> ,s.c_id
-> ,s.s_score
-> ,t.avg_score
-> from Score s
-> join (select
->     s_id                中间表
->     ,round(avg(s_score),2) avg_score
->     from Score
->     group by s_id)t
-> on s.s_id = t.s_id
-> order by 4 desc;
```

```
+-----+-----+-----+-----+
| s_id | c_id | s_score | avg_score |
+-----+-----+-----+-----+
| 07 | 02 | 89 | 91.50 |
| 07 | 03 | 94 | 91.50 |
| 01 | 01 | 80 | 88.67 |
| 01 | 02 | 90 | 88.67 |
| 01 | 03 | 96 | 88.67 |
| 03 | 01 | 80 | 82.00 |
| 03 | 02 | 81 | 82.00 |
| 03 | 03 | 85 | 82.00 |
| 05 | 01 | 76 | 81.50 |
| 05 | 02 | 87 | 81.50 |
| 02 | 02 | 60 | 70.00 |
| 02 | 03 | 80 | 70.00 |
| 02 | 01 | 70 | 70.00 |
| 06 | 01 | 43 | 49.50 |
| 06 | 03 | 56 | 49.50 |
| 04 | 01 | 50 | 40.00 |
| 04 | 02 | 40 | 40.00 |
| 04 | 03 | 30 | 40.00 |
+-----+-----+-----+-----+
18 rows in set (0.00 sec)
```

```
1 -- 参考方法1
2 select
3     s.s_id
4     ,(select s_score from Score where s_id=s.s_id and c_id='01')
5     as 语文
6     ,(select s_score from Score where s_id=s.s_id and c_id='02')
7     as 数学
8     ,(select s_score from Score where s_id=s.s_id and c_id='03')
9     as 英语
```

```

10      ,round(avg(s_score),2) 平均分
11  from Score s
12  group by s.s_id
13  order by 5 desc;

```

```

mysql> select
-> s.s_id
-> ,(select s_score from Score where s_id=s.s_id and c_id='01')
-> as 语文
-> ,(select s_score from Score where s_id=s.s_id and c_id='02')
-> as 数学
-> ,(select s_score from Score where s_id=s.s_id and c_id='03')
-> as 英语
-> ,round(avg(s_score),2) 平均分
-> from Score s
-> group by s.s_id
-> order by 5 desc;

```

```

+-----+-----+-----+-----+
| s_id | 语文 | 数学 | 英语 | 平均分 |
+-----+-----+-----+-----+
| 07 | NULL | 89 | 94 | 91.50 |
| 01 | 80 | 90 | 96 | 88.67 |
| 03 | 80 | 81 | 85 | 82.00 |
| 05 | 76 | 87 | NULL | 81.50 |
| 02 | 70 | 60 | 80 | 70.00 |
| 06 | 43 | NULL | 56 | 49.50 |
| 04 | 50 | 40 | 30 | 40.00 |
+-----+-----+-----+-----+
7 rows in set (0.00 sec)

```

NB: 一张表Score表搞定全部

```

1  select
2      s.s_id
3      ,max(case s.c_id when '01' then s.s_score end) 语文
4      ,max(case s.c_id when '02' then s.s_score end) 数学
5      ,max(case s.c_id when '03' then s.s_score end) 英语
6      ,avg(s.s_score)
7      ,b.s_name -- 没有出现在group by子句中，导致报错
8  join Student b
9  on s.s_id = b.s_id
10 group by s.s_id
11 order by 5 desc;

```

严格模式的报错:

ERROR 1055 (42000): Expression #6 of SELECT list is not in GROUP BY clause and contains nonaggregated column "test.b.s_name" which is not functionally dependent on columns in GROUP BY clause; this is incompatible with sql_mode=only_full_group_by

对于 **GROUP BY** 聚合操作，如果在 **SELECT** 中的列，没有在 **GROUP BY** 中出现，那么这个 **SQL** 是不合法的，因为列不在 **GROUP BY** 从句中，也就是说查出来的列必须在 **group by** 后面出现否则就会报错，或者这个字段出现在聚合函数里面。

```

1  -- 参考方法2：将上面的b.s_name去掉
2
3  select

```

```

4      s.s_id
5      ,max(case s.c_id when '01' then s.s_score end) 语文
6      ,max(case s.c_id when '02' then s.s_score end) 数学
7      ,max(case s.c_id when '03' then s.s_score end) 英语
8      ,round(avg(s.s_score),2) avg_score
9  from Score s
10 join Student b
11 on s.s_id = b.s_id
12 group by s.s_id
13 order by 5 desc;

```

```

mysql> select
->      s.s_id
->      ,max(case s.c_id when '01' then s.s_score end) 语文
->      ,max(case s.c_id when '02' then s.s_score end) 数学
->      ,max(case s.c_id when '03' then s.s_score end) 英语
->      ,round(avg(s.s_score),2) avg_score
->  from Score s
->  join Student b
->  on s.s_id = b.s_id
->  group by s.s_id
->  order by 5 desc;
+-----+-----+-----+-----+
| s_id | 语文 | 数学 | 英语 | avg_score |
+-----+-----+-----+-----+
| 07 | NULL | 89 | 94 | 91.50 |
| 01 | 80 | 90 | 96 | 88.67 |
| 03 | 80 | 81 | 85 | 82.00 |
| 05 | 76 | 87 | NULL | 81.50 |
| 02 | 70 | 60 | 80 | 70.00 |
| 06 | 43 | NULL | 56 | 49.50 |
| 04 | 50 | 40 | 30 | 40.00 |
+-----+-----+-----+-----+
7 rows in set (0.00 sec)

```

1.19 题目 18

1.19.1 题目需求

查询各科成绩最高分、最低分和平均分：以如下形式显示：课程 ID，课程 name，最高分，最低分，平均分，及格率，中等率，优良率，优秀率；及格：>=60，中等为：70-80，优良为：80-90，优秀为：>=90

1.19.2 SQL 实现

思路清晰：统计每个阶段的总人数，再除以总共的人数即可

将成绩表和课程表联合起来进行查询：

- case 语句用于对每个分数贴标签

- sum 语句对相应的语句中的 1 进行求和

```

1  select
2      s.c_id
3      ,c.c_name
4      ,max(s.s_score)
5      ,min(s.s_score)
6      ,round(avg(s.s_score), 2)
7      ,round(100 * (sum(case when s.s_score >= 60 then 1 else 0 end) / sum(case
8          when s.s_score then 1 else 0 end)), 2) as 及格率
9      ,round(100 * (sum(case when s.s_score >= 70 and s.s_score <= 80 then 1 else
10         0 end) / sum(case when s.s_score then 1 else 0 end)), 2) as 中等率
11     ,round(100 * (sum(case when s.s_score >= 80 and s.s_score <= 90 then 1 else
12         0 end) / sum(case when s.s_score then 1 else 0 end)), 2) as 优良率
13     ,round(100 * (sum(case when s.s_score >= 90 then 1 else 0 end) / sum(case
14         when s.s_score then 1 else 0 end)), 2) as 优秀率
15 from Score s
16 left join Course c
17 on s.c_id = c.c_id
18 group by s.c_id, c.c_name;

```

```

mysql> select
-> s.c_id
-> ,c.c_name
-> ,max(s.s_score)
-> ,min(s.s_score)
-> ,round(avg(s.s_score), 2)
-> ,round(100 * (sum(case when s.s_score >= 60 then 1 else 0 end) / sum(case when s.s_score then 1 else 0 end)), 2) as 及格率
-> ,round(100 * (sum(case when s.s_score >= 70 and s.s_score <= 80 then 1 else 0 end) / sum(case when s.s_score then 1 else 0 end)), 2) as
中等率
-> ,round(100 * (sum(case when s.s_score >= 80 and s.s_score <= 90 then 1 else 0 end) / sum(case when s.s_score then 1 else 0 end)), 2) as
优良率
-> ,round(100 * (sum(case when s.s_score >= 90 then 1 else 0 end) / sum(case when s.s_score then 1 else 0 end)), 2) as 优秀率
-> from Score s
-> left join Course c
-> on s.c_id = c.c_id
-> group by s.c_id, c.c_name;
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| c_id | c_name | max(s.s_score) | min(s.s_score) | round(avg(s.s_score), 2) | 及格率 | 中等率 | 优良率 | 优秀率 |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 01 | 语文 | 88 | 43 | 66.50 | 66.67 | 66.67 | 33.33 | 0.00 |
| 02 | 数学 | 98 | 49 | 74.50 | 83.33 | 0.00 | 66.67 | 16.67 |
| 03 | 英语 | 96 | 30 | 73.50 | 66.67 | 16.67 | 33.33 | 33.33 |
+-----+-----+-----+-----+-----+-----+-----+-----+
3 rows in set (0.00 sec)

```

1.20 题目 19

1.20.1 题目需求

按照各科成绩进行排序，并且显示排名

1.20.2 分析过程

题目的意思是：将每科的成绩单独进行排名，类似如下的效果：

课程名	分数	排名
英语	99	1

课程名	分数	排名
英语	92	2
英语	89	3
数学	88	1
数学	85	2
.....		

1.20.3 SQL 实现

第一步：我们对 Score 表中的一门课程进行排名，比如 01 课程

```
1 select * from(  
2     select  
3         t1.c_id -- 课程号  
4         ,t1.s_score -- 分数  
5         ,(select count(distinct t2.s_score) -- 课程去重  
6           from Score t2  
7           where t2.s_score >= t1.s_score -- SQL 实现 排序  
8             and t2.c_id = '01') rank  
9     from Score t1 -- 通过相同的表实现自连接  
10    where t1.c_id = '01'  
11    order by t1.s_score desc  
12 )t1
```

上面是针对 01 课程，结果为：

```
mysql> select * from(
->   select
->   t1.c_id -- 课程号
->   ,t1.s_score -- 分数
->   ,(select count(distinct t2.s_score) -- 课程去重
->     from Score t2
->     where t2.s_score >= t1.s_score -- SQL实现排序
->     and t2.c_id = '01') rank
->   from Score t1 -- 通过相同的表实现自连接
->   where t1.c_id = '01'
->   order by t1.s_score desc
-> )t1;
+-----+-----+-----+
| c_id | s_score | rank |
+-----+-----+-----+
| 01   |      80 |    1 |
| 01   |      80 |    1 |
| 01   |      76 |    2 |
| 01   |      70 |    3 |
| 01   |      50 |    4 |
| 01   |      43 |    5 |
+-----+-----+-----+
6 rows in set (0.00 sec)
```

SQL实现排序

第二步：我们将 01、02、03 课程全部连接起来，通过 union 实现

- 表的自连接
- SQL 实现排序

```
1  -- 自己的方法
2
3  select * from(
4    select
5      t1.c_id -- 课程号
6      ,t1.s_score -- 分数
7      ,(select count(distinct t2.s_score) -- 课程去重
8        from Score t2
9        where t2.s_score >= t1.s_score -- SQL实现排序
10       and t2.c_id = '01') rank
11    from Score t1 -- 通过相同的表实现自连接
12    where t1.c_id = '01'
13    order by t1.s_score desc
14  )t1
15
16  union
17  select * from(
18    select
19      t1.c_id -- 课程号
20      ,t1.s_score -- 分数
21      ,(select count(distinct t2.s_score) -- 课程去重
22        from Score t2
23        where t2.s_score >= t1.s_score -- SQL实现排序
```



```
24         and t2.c_id = '02') rank
25     from Score t1    -- 通过相同的表实现自连接
26     where t1.c_id = '02'
27     order by t1.s_score desc
28 )t2
29
30 union
31 select * from(
32     select
33         t1.c_id -- 课程号
34         ,t1.s_score -- 分数
35         ,(select count(distinct t2.s_score)
36            from Score t2
37            where t2.s_score >= t1.s_score
38            and t2.c_id = '03') rank
39     from Score t1
40     where t1.c_id = '03'
41     order by t1.s_score desc
42 )t3;
```

c_id	s_score	rank
01	80	1
01	76	2
01	70	3
01	50	4
01	43	5
02	90	1
02	89	2
02	87	3
02	81	4
02	60	5
02	40	6
03	96	1
03	94	2
03	85	3
03	80	4
03	56	5
03	30	6

```

1  -- 参考代码
2
3  select * from (select
4      t1.c_id,
5      t1.s_score,
6      (select count(distinct t2.s_score)
7          from Score t2
8          where t2.s_score>=t1.s_score and t2.c_id='01') rank
9  from Score t1 where t1.c_id='01'
10 order by t1.s_score desc) t1
11

```

```
12 union
13 select * from (select
14                 t1.c_id
15                 ,t1.s_score
16                 ,(select count(distinct t2.s_score)
17                   from Score t2
18                   where t2.s_score>=t1.s_score and t2.c_id='02') rank
19                 from Score t1 where t1.c_id='02'
20                 order by t1.s_score desc) t2
21
22 union
23 select * from (select
24                 t1.c_id,
25                 t1.s_score,
26                 (select count(distinct t2.s_score) from Score t2 where t2.
27                   s_score>=t1.s_score and t2.c_id='03') rank
28                 from Score t1 where t1.c_id='03'
29                 order by t1.s_score desc) t3
```

1.21 题目 20

1.21.1 题目需求

查询学生的总成绩，并进行排名

1.21.2 SQL 实现

1、先查询每个学生的总成绩

```
1 select
2     s_id
3     ,sum(s_score)
4 from Score
5 group by s_id
6 order by 2 desc;
```

```
mysql> select
      -> s_id
      -> ,sum(s_score)
      -> from Score
      -> group by s_id
      -> order by 2 desc;
+-----+-----+
| s_id | sum(s_score) |
+-----+-----+
| 01   |          266 |
| 03   |          246 |
| 02   |          210 |
| 07   |          183 |
| 05   |          163 |
| 04   |          120 |
| 06   |           99 |
+-----+-----+
7 rows in set (0.00 sec)
```

将上面的结果和学生信息表进行关联查询：

```
1  --
2  select
3      s.s_name
4      ,s.s_id
5      ,t.score
6  from Student s
7  join(select
8      s_id
9      ,sum(s_score) score
```

```

10     from Score
11     group by s_id
12     order by 2 desc
13 )t
14 on s.s_id = t.s_id;

```

```

mysql> select
-> s.s_name
-> ,s.s_id
-> ,t.score
-> from Student s
-> join(select
->     s_id
->     ,sum(s_score) score
->     from Score
->     group by s_id
->     order by 2 desc
-> )t
-> on s.s_id = t.s_id;
+-----+-----+-----+
| s_name | s_id | score |
+-----+-----+-----+
| 赵雷   | 01   | 266   |
| 孙风   | 03   | 246   |
| 钱电   | 02   | 210   |
| 郑竹   | 07   | 183   |
| 周梅   | 05   | 163   |
| 李云   | 04   | 120   |
| 吴兰   | 06   | 99    |
+-----+-----+-----+
7 rows in set (0.00 sec)

```

```

1 -- 不使用中间表查询
2
3 select
4     s.s_id
5     ,s.s_name
6     ,sum(sc.s_score) score
7 from Student s
8 join Score sc

```



```

6      from(select s.s_id, s.s_name, sum(sc.s_score) score
7          from Student s
8          join Score sc
9          on s.s_id = sc.s_id
10         group by s.s_id
11         order by 3 desc)t2  -- t2和t1相同
12     where t2.score > t1.score) + 1 as rank
13 from(
14     select s.s_id ,s.s_name ,sum(sc.s_score) score
15     from Student s
16     join Score sc
17     on s.s_id = sc.s_id
18     group by s.s_id
19     order by 3 desc)t1  -- t1
20 order by 3 desc;

```

```

mysql> select
-> t1.s_id ,t1.s_name, t1.score
-> ,(select count(t2.score)
->     from select s.s_id, s.s_name, sum(sc.s_score) score
->     from Student s
->     join Score sc
->     on s.s_id = sc.s_id
->     group by s.s_id
->     order by 3 desc)t2  -- t2和t1相同
->     where t2.score > t1.score) + 1 as rank
-> from(
->     select s.s_id ,s.s_name ,sum(sc.s_score) score
->     from Student s
->     join Score sc
->     on s.s_id = sc.s_id
->     group by s.s_id
->     order by 3 desc)t1  -- t1
-> order by 3 desc;
+-----+-----+-----+-----+
| s_id | s_name | score | rank |
+-----+-----+-----+-----+
| 01   | 赵雷   | 266   | 1    |
| 03   | 孙风   | 246   | 2    |
| 02   | 钱电   | 210   | 3    |
| 07   | 郑竹   | 183   | 4    |
| 05   | 周梅   | 163   | 5    |
| 04   | 李云   | 120   | 6    |
| 06   | 吴兰   | 99    | 7    |
+-----+-----+-----+-----+
7 rows in set (0.00 sec)

```

1.22 题目 21

1.22.1 题目需求

查询不同老师所教不同课程平均分从高到低显示

1.22.2 SQL 实现

先找出每个老师教授了哪些课程：

```
1 select
2     c.c_name
3     ,t.t_name
4 from Course c
5 left join Teacher t
6 on c.t_id = t.t_id;
```

```
mysql> select
-> c.c_name
-> ,t.t_name
-> from Course c
-> left join Teacher t
[ -> on c.t_id = t.t_id;
+-----+-----+
| c_name | t_name |
+-----+-----+
| 数学   | 张三   |
| 语文   | 李四   |
| 英语   | 王五   |
+-----+-----+
3 rows in set (0.00 sec)
```

将上面的结果和成绩表连接起来：

```
1 select
```



```
2      c.c_name
3      ,t.t_name
4      ,round(avg(s.s_score),2)  score    -- 课程分组后再求均值
5  from Course c    -- 主表，通过两次连接
6  left join Teacher t
7  on c.t_id = t.t_id
8  left join Score s
9  on c.c_id = s.c_id
10 group by c.c_id    -- 课程分组
11 order by 3 desc;   -- 降序
```

```
mysql> select
-> c.c_name
-> ,t.t_name
-> ,round(avg(s.s_score),2)  score    -- 课程分组后再求均值
-> from Course c    -- 主表，通过两次连接
-> left join Teacher t
-> on c.t_id = t.t_id
-> left join Score s
-> on c.c_id = s.c_id
-> group by c.c_id
-> order by 3 desc; -- 降序
+-----+-----+-----+
| c_name | t_name | score |
+-----+-----+-----+
| 数学   | 张三   | 74.50 |
| 英语   | 王五   | 73.50 |
| 语文   | 李四   | 66.50 |
+-----+-----+-----+
3 rows in set (0.00 sec)
```

1.23 题目 22

1.23.1 题目需求

查询所有课程的成绩第 2 名到第 3 名的学生信息及该课程成绩

1.23.2 SQL 实现

1.23.2.1 自己的方法

1、课程表和成绩表连接起来，显示所有的课程和成绩信息

```
1  select
2      s.s_id
3      ,s.c_id
4      ,s.s_score
```

```
5     ,c.c_name
6 from Score s
7 join Course c
8 on s.c_id = c.c_id
```

```
mysql> select
-> s.s_id
-> ,s.c_id
-> ,s.s_score
-> ,c.c_name
-> from Score s
-> join Course c
-> on s.c_id = c.c_id;
+-----+-----+-----+-----+
| s_id | c_id | s_score | c_name |
+-----+-----+-----+-----+
| 01   | 01   | 80      | 语文   |
| 01   | 02   | 90      | 数学   |
| 01   | 03   | 96      | 英语   |
| 02   | 01   | 70      | 语文   |
| 02   | 02   | 60      | 数学   |
| 02   | 03   | 80      | 英语   |
| 03   | 01   | 80      | 语文   |
| 03   | 02   | 81      | 数学   |
| 03   | 03   | 85      | 英语   |
| 04   | 01   | 50      | 语文   |
| 04   | 02   | 40      | 数学   |
| 04   | 03   | 30      | 英语   |
| 05   | 01   | 76      | 语文   |
| 05   | 02   | 87      | 数学   |
| 06   | 01   | 43      | 语文   |
| 06   | 03   | 56      | 英语   |
| 07   | 02   | 89      | 数学   |
| 07   | 03   | 94      | 英语   |
+-----+-----+-----+-----+
18 rows in set (0.01 sec)
```

2、查出全部的语文成绩

```
1 select s.s_id, s.s_score, c.c_name
2 from Score s
```

```
3 join Course c on s.c_id = c.c_id
4 where c.c_name = '语文'
5 order by s.s_score desc;
```

```
mysql> select s.s_id, s.s_score, c.c_name
-> from Score s
-> join Course c on s.c_id = c.c_id
-> where c.c_name = '语文'
[ -> order by s.s_score desc;
+-----+-----+-----+
| s_id | s_score | c_name |
+-----+-----+-----+
| 01   |      80 | 语文   |
| 03   |      80 | 语文   |
| 05   |      76 | 语文   |
| 02   |      70 | 语文   |
| 04   |      50 | 语文   |
| 06   |      43 | 语文   |
+-----+-----+-----+
6 rows in set (0.00 sec)
```

3、我们找出语文的第 2、3 的学生

```
1 select s.s_id, s.s_score, c.c_name
2 from Score s
3 join Course c on s.c_id = c.c_id
4 where c.c_name = '语文'
5 order by s.s_score desc
6 limit 1, 2;
```

```
mysql> select s.s_id, s.s_score, c.c_name
-> from Score s
-> join Course c on s.c_id = c.c_id
-> where c.c_name = '语文'
-> order by s.s_score desc
-> limit 1,2;
```

从第1行数据之后取2行，
即第2、3行数据

s_id	s_score	c_name
03	80	语文
05	76	语文

2 rows in set (0.00 sec)

4、同时求出语文、数学、英语的分数，并且通过union拼接

```
1  -- union 连接
2
3  (select s.s_id, s.s_score, c.c_name
4  from Score s
5  join Course c on s.c_id = c.c_id
6  where c.c_name = '语文'
7  order by s.s_score desc
8  limit 1, 2)
9
10 union
11
12 (select s.s_id, s.s_score, c.c_name
13 from Score s
14 join Course c on s.c_id = c.c_id
15 where c.c_name = '数学'
16 order by s.s_score desc
17 limit 1, 2)
18
19 union
20 ((select s.s_id, s.s_score, c.c_name
21 from Score s
22 join Course c on s.c_id = c.c_id
23 where c.c_name = '英语'
24 order by s.s_score desc
25 limit 1, 2))
```

s_id	s_score	c_name
03	80	语文
05	76	语文
07	89	数学
05	87	数学
07	94	英语
03	85	英语

6 rows in set (0.00 sec)

5、将上面的结果学生信息表进行连接即可

好歹是实现了 ☹

```

1  -- 最终脚本
2  -- !!!!! 真的需要好好优化下
3
4  select
5      s.s_id
6      ,s.s_name
7      ,t.c_name
8      ,t.s_score
9  from Student s
10 join (-- union连接
11
12 (select s.s_id, s.s_score, c.c_name
13  from Score s
14  join Course c on s.c_id = c.c_id
15  where c.c_name = '语文'
16  order by s.s_score desc
17  limit 1, 2)
18
19 union
20
21 (select s.s_id, s.s_score, c.c_name

```

```

22 from Score s
23 join Course c on s.c_id = c.c_id
24 where c.c_name = '数学'
25 order by s.s_score desc
26 limit 1, 2)
27
28 union
29 ((select s.s_id, s.s_score, c.c_name
30 from Score s
31 join Course c on s.c_id = c.c_id
32 where c.c_name = '英语'
33 order by s.s_score desc
34 limit 1, 2)))t -- 临时表 t
35
36 on s.s_id = t.s_id

```

```

+-----+-----+-----+-----+
| s_id | s_name | c_name | s_score |
+-----+-----+-----+-----+
| 03   | 孙风   | 语文   | 80      |
| 05   | 周梅   | 语文   | 76      |
| 07   | 郑竹   | 数学   | 89      |
| 05   | 周梅   | 数学   | 87      |
| 07   | 郑竹   | 英语   | 94      |
| 03   | 孙风   | 英语   | 85      |
+-----+-----+-----+-----+
6 rows in set (0.00 sec)

```

1.23.2.2 和第 25 题相同的方法

1、以语文为例，首先我们找出前 3 名的成绩（包含相同的成绩）

```

1 -- 语文
2 select
3     a.s_id
4     ,a.c_id
5     ,a.s_score -- 3、此时a表的成绩就是我们找的
6 from Score a
7 join Score b
8 on a.c_id = b.c_id
9 and a.s_score <= b.s_score -- 1、判断a的分数小于等于b的分数，要带上等号

```

```

10 and a.c_id="01"
11 group by 1,2
12 having count(b.s_id) <= 3 -- 2、b中的个数至少有3个，应对分数相同的情形
13 order by 3 desc
14 limit 1,2

```

```

mysql> select
-> a.s_id
-> ,a.c_id
-> ,a.s_score -- 3、此时a表的成绩就是我们找的
-> from Score a
-> join Score b
-> on a.c_id = b.c_id
-> and a.s_score <= b.s_score -- 1、判断a的分数小于等于b的分数，要带上等号
-> and a.c_id="01" 带上=，下面就会出现相同的分数
-> group by 1,2
-> having count(b.s_id) <= 3 -- 2、b中的个数至少有3个，应对分数相同的情形
-> order by 3 desc;
+-----+-----+-----+
| s_id | c_id | s_score |
+-----+-----+-----+
| 01 | 01 | 80 |
| 03 | 01 | 80 |
| 05 | 01 | 76 |
+-----+-----+-----+
3 rows in set (0.00 sec)

```

```

1 -- 语文
2 select
3     a.s_id
4     ,a.c_id
5     ,a.s_score -- 3、此时a表的成绩就是我们找的
6 from Score a
7 join Score b
8 on a.c_id = b.c_id
9 and a.s_score <= b.s_score -- 1、判断a的分数小于等于b的分数，要带上等号
10 and a.c_id="01"
11 group by 1,2
12 having count(b.s_id) <= 3 -- 2、b中的个数至少有3个，应对分数相同的情形
13 order by 3 desc
14 limit 1,2; -- 取得第2、3名

```

在通过数学和英语的类似操作得到 2、3 名的成绩，再进行拼接即可

1.24 题目 23

1.24.1 题目需求

统计各科成绩各分数段人数：课程编号，课程名称，[100-85]，[85-70]，[70-60]，[0-60] 及所占百分比

1.24.2 SQL 实现

1.24.2.1 自己的方法

1、如何对每个成绩进行分组展示：ABCD 代表相应的等级

```
1 select
2     c_id
3     ,s_score
4     ,case when s_score >= 85 and s_score<= 100 then 'A'    -- 大小关系必须分两
        次写，一次写的话MySQL无法识别
5         when 70 <= s_score and s_score < 85 then 'B'
6         when 60 <= s_score and s_score < 70 then 'C'
7         when 0 <= s_score and s_score < 60 then 'D'
8         else '其他' end as 'category'
9 from Score s;
```


c_id	s_score	category
01	80	B
02	90	A
03	96	A
01	70	B
02	60	C
03	80	B
01	80	B
02	81	B
03	85	A
01	50	D
02	40	D
03	30	D
01	76	B
02	87	A
01	43	D
03	56	D
02	89	A
03	94	A

18 rows in set (0.00 sec)

2、将两个表关联起来展示数据

```

1  -- 1、查看全部课程和成绩信息
2
3  select
4      s.c_id
5      ,c.c_name
6      ,s.s_score
7      ,case when s.s_score >= 85 and s.s_score<= 100 then 'A'  -- 大小关系必须

```

```

      分两次写，一次写的话MySQL无法识别
8      when 70 <= s.s_score and s.s_score < 85 then 'B'
9      when 60 <= s.s_score and s.s_score < 70 then 'C'
10     when 0 <= s.s_score and s.s_score < 60 then 'D'
11     else '其他' end as 'category'
12 from Score s
13 join Course c
14 on s.c_id = c.c_id;

```

c_id	c_name	s_score	category
01	语文	80	B
02	数学	90	A
03	英语	96	A
01	语文	70	B
02	数学	60	C
03	英语	80	B
01	语文	80	B
02	数学	81	B
03	英语	85	A
01	语文	50	D
02	数学	40	D
03	英语	30	D
01	语文	76	B
02	数学	87	A
01	语文	43	D
03	英语	56	D
02	数学	89	A
03	英语	94	A

3、完整代码

```

1 select
2     s.c_id 编号
3     ,c.c_name 科目

```

```

4      ,sum(case when s.s_score >= 85 and s.s_score<= 100 then 1 else 0 end) "
      [85,100]人数"
5      ,round(100 * (sum(case when s.s_score >= 85 and s.s_score<= 100 then 1
      else 0 end) / sum(case when s.s_score then 1 else 0 end)), 2) as '
      [85,100]百分比'
6      ,sum(case when s.s_score >= 70 and s.s_score<= 85 then 1 else 0 end) "
      [70,85]人数"
7      ,round(100 * (sum(case when s.s_score >= 70 and s.s_score<= 85 then 1 else
      0 end) / sum(case when s.s_score then 1 else 0 end)), 2) as '[70,85]百
      分比'
8      ,sum(case when s.s_score >= 60 and s.s_score<= 70 then 1 else 0 end) "
      [60,70]人数"
9      ,round(100 * (sum(case when s.s_score >= 60 and s.s_score<= 70 then 1 else
      0 end) / sum(case when s.s_score then 1 else 0 end)), 2) as '[60,70]百
      分比'
10     ,sum(case when s.s_score >= 0 and s.s_score<= 60 then 1 else 0 end) "
      [0,60]人数"
11     ,round(100 * (sum(case when s.s_score >= 0 and s.s_score<= 60 then 1 else
      0 end) / sum(case when s.s_score then 1 else 0 end)), 2) as '[0,60]百分
      比'
12 from Score s
13 left join Course c
14 on s.c_id = c.c_id
15 group by s.c_id, c.c_name

```

```

mysql> select
-> .s.c_id 编号
-> .c.c_name 科目
-> sum(case when s.s_score >= 85 and s.s_score<= 100 then 1 else 0 end) "[85,100]人数"
-> sum(case when s.s_score >= 85 and s.s_score<= 100 then 1 else 0 end) / sum(case when s.s_score then 1 else 0 end), 2) as '[85,100]百分比'
-> sum(case when s.s_score >= 70 and s.s_score<= 85 then 1 else 0 end) "[70,85]人数"
-> round(100 * (sum(case when s.s_score >= 70 and s.s_score<= 85 then 1 else 0 end) / sum(case when s.s_score then 1 else 0 end)), 2) as '[70,85]百分比'
-> sum(case when s.s_score >= 60 and s.s_score<= 70 then 1 else 0 end) "[60,70]人数"
-> round(100 * (sum(case when s.s_score >= 60 and s.s_score<= 70 then 1 else 0 end) / sum(case when s.s_score then 1 else 0 end)), 2) as '[60,70]百分比'
-> sum(case when s.s_score >= 0 and s.s_score<= 60 then 1 else 0 end) "[0,60]人数"
-> round(100 * (sum(case when s.s_score >= 0 and s.s_score<= 60 then 1 else 0 end) / sum(case when s.s_score then 1 else 0 end)), 2) as '[0,60]百分比'
-> from Score s
-> left join Course c
-> on s.c_id = c.c_id
-> group by s.c_id, c.c_name;

```

编号	科目	[85,100]人数	[85,100]百分比	[70,85]人数	[70,85]百分比	[60,70]人数	[60,70]百分比	[0,60]人数	[0,60]百分比
01	语文	0	0.00	4	66.67	1	16.67	2	33.33
02	数学	3	50.00	1	16.67	1	16.67	2	33.33
03	英语	3	50.00	2	33.33	0	0.00	2	33.33

3 rows in set (0.00 sec)

1.24.2.2 参考方法

1、先统计每个阶段的人数和占比

```

1 select
2     c_id
3     ,sum(case when s_score > 85 and s_score <=100 then 1 else 0 end) as '85-100
      '
4     ,round(100 * (sum(case when s_score > 85 and s_score <= 100 then 1 else 0
      end) / count(*)), 2) '占比'
5 from Score
6 group by c_id; -- 分课程统计总数和占比
7
8

```

```

9  -- 方式2
10 select
11     c_id
12     ,sum(case when s_score > 85 and s_score <=100 then 1 else 0 end) as '85-100'
13     ,round(100 * (sum(case when s_score > 85 and s_score <= 100 then 1 else 0
14         end) / count(case when s_score then 1 else 0 end)), 2) '占比' -- 不同
15     count(*)
16 from Score
17 group by c_id;

```

```

mysql> select
-> c_id
-> ,sum(case when s_score > 85 and s_score <=100 then 1 else 0 end) as '85-100'
-> ,round(100 * (sum(case when s_score > 85 and s_score <= 100 then 1 else 0 end) / count(*)), 2) '占比'
-> from Score
-> group by c_id;
+-----+-----+
| c_id | 85-100 | 占比 |
+-----+-----+
| 01 | 0 | 0.00 |
| 02 | 3 | 50.00 |
| 03 | 2 | 33.33 |
+-----+-----+
3 rows in set (0.00 sec)

```

注意对比:

```

mysql> select
-> c_id
-> ,sum(case when s_score > 85 and s_score <=100 then 1 else 0 end) as '85-100'
-> ,round(100 * (sum(case when s_score > 85 and s_score <= 100 then 1 else 0 end) / count(case when s_score then 1 else 0 end)), 2) '占比'
-- 不同count(*)
-> from Score
-> group by c_id;
+-----+-----+
| c_id | 85-100 | 占比 |
+-----+-----+
| 01 | 0 | 0.00 |
| 02 | 3 | 50.00 |
| 03 | 2 | 33.33 |
+-----+-----+
3 rows in set (0.00 sec)

```

2、我们将 4 种情况同时查出来

```

1 select
2     c_id
3     ,sum(case when s_score > 85 and s_score <=100 then 1 else 0 end) as '85-100'
4     ,round(100 * (sum(case when s_score > 85 and s_score <= 100 then 1 else 0
5         end) / count(*)), 2) '[85,100] 占比'
6     ,sum(case when s_score > 70 and s_score <=85 then 1 else 0 end) as '70-85'
7     ,round(100 * (sum(case when s_score > 70 and s_score <= 85 then 1 else 0
8         end) / count(*)), 2) '[70,85] 占比'
9     ,sum(case when s_score > 60 and s_score <=70 then 1 else 0 end) as '60-70'
10    ,round(100 * (sum(case when s_score > 60 and s_score <= 70 then 1 else 0
11        end) / count(*)), 2) '[60,70] 占比'
12    ,sum(case when s_score >=0 and s_score <=60 then 1 else 0 end) as '0-60'
13    ,round(100 * (sum(case when s_score > 0 and s_score <= 60 then 1 else 0 end
14        ) / count(*)), 2) '[0,60] 占比'
15 from Score
16 group by c_id; -- 分课程统计总数和占比

```

```
mysql> select
-> c_id
-> ,sum(case when s_score > 85 and s_score <=100 then 1 else 0 end) as '85-100'
-> ,round(100 * (sum(case when s_score > 85 and s_score <= 100 then 1 else 0 end) / count(*)), 2) '[85,100]占比'
-> ,sum(case when s_score > 70 and s_score <=85 then 1 else 0 end) as '70-85'
-> ,round(100 * (sum(case when s_score > 70 and s_score <= 85 then 1 else 0 end) / count(*)), 2) '[70,85]占比'
-> ,sum(case when s_score > 60 and s_score <=70 then 1 else 0 end) as '60-70'
-> ,round(100 * (sum(case when s_score > 60 and s_score <= 70 then 1 else 0 end) / count(*)), 2) '[60,70]占比'
-> ,sum(case when s_score >=0 and s_score <=60 then 1 else 0 end) as '0-60'
-> ,round(100 * (sum(case when s_score > 0 and s_score <= 60 then 1 else 0 end) / count(*)), 2) '[0,60]占比'
-> from Score
-> group by c_id; -- 分课程统计总数和占比
```

c_id	85-100	[85,100]占比	70-85	[70,85]占比	60-70	[60,70]占比	0-60	[0,60]占比
01	0	0.00	3	50.00	1	16.67	2	33.33
02	3	50.00	1	16.67	0	0.00	2	33.33
03	2	33.33	2	33.33	0	0.00	2	33.33

3 rows in set (0.00 sec)

3、将科目名称连接起来

```
1 -- 整体和自己的方法是类似的
2 select
3     s.c_id
4     ,c.c_name
5     ,sum(case when s_score > 85 and s_score <=100 then 1 else 0 end) as '85-100'
6     ,round(100 * (sum(case when s_score > 85 and s_score <= 100 then 1 else 0
7         end) / count(*)), 2) '[85,100]占比'
8     ,sum(case when s_score > 70 and s_score <=85 then 1 else 0 end) as '70-85'
9     ,round(100 * (sum(case when s_score > 70 and s_score <= 85 then 1 else 0
10        end) / count(*)), 2) '[70,85]占比'
11    ,sum(case when s_score > 60 and s_score <=70 then 1 else 0 end) as '60-70'
12    ,round(100 * (sum(case when s_score > 60 and s_score <= 70 then 1 else 0
13        end) / count(*)), 2) '[60,70]占比'
14    ,sum(case when s_score >=0 and s_score <=60 then 1 else 0 end) as '0-60'
15    ,round(100 * (sum(case when s_score > 0 and s_score <= 60 then 1 else 0 end
16        ) / count(*)), 2) '[0,60]占比'
17 from Score s
18 left join Course c
19 on s.c_id = c.c_id
20 group by s.c_id, c.c_name; -- 分课程统计总数和占比
```

```
mysql> select
-> s.c_id
-> ,c.c_name
-> ,sum(case when s_score > 85 and s_score <=100 then 1 else 0 end) as '85-100'
-> ,round(100 * (sum(case when s_score > 85 and s_score <= 100 then 1 else 0 end) / count(*)), 2) '[85,100]占比'
-> ,sum(case when s_score > 70 and s_score <=85 then 1 else 0 end) as '70-85'
-> ,round(100 * (sum(case when s_score > 70 and s_score <= 85 then 1 else 0 end) / count(*)), 2) '[70,85]占比'
-> ,sum(case when s_score > 60 and s_score <=70 then 1 else 0 end) as '60-70'
-> ,round(100 * (sum(case when s_score > 60 and s_score <= 70 then 1 else 0 end) / count(*)), 2) '[60,70]占比'
-> ,sum(case when s_score >=0 and s_score <=60 then 1 else 0 end) as '0-60'
-> ,round(100 * (sum(case when s_score > 0 and s_score <= 60 then 1 else 0 end) / count(*)), 2) '[0,60]占比'
-> from Score s
-> left join Course c
-> on s.c_id = c.c_id
-> group by s.c_id, c.c_name; -- 分课程统计总数和占比
```

c_id	c_name	85-100	[85,100]占比	70-85	[70,85]占比	60-70	[60,70]占比	0-60	[0,60]占比
01	语文	0	0.00	3	50.00	1	16.67	2	33.33
02	数学	3	50.00	1	16.67	0	0.00	2	33.33
03	英语	2	33.33	2	33.33	0	0.00	2	33.33

3 rows in set (0.00 sec)

1.25 题目 24

1.25.1 题目需求

查询学生的平均成绩及名次

1.25.2 SQL 实现

1.25.2.1 自己的方法

1、先求出每个人的平均分

```
1  -- 自己的方法
2
3  select
4      sc.s_id
5      ,s.s_name
6      ,round(avg(sc.s_score),2)  avg_score
7  from Score sc
8  join Student s
9  on sc.s_id=s.s_id
10 group by sc.s_id,s.s_name
```

```
mysql> select
-> sc.s_id
-> ,s.s_name
-> ,round(avg(sc.s_score),2) avg_score
-> from Score sc
-> join Student s
-> on sc.s_id=s.s_id
-> group by sc.s_id,s.s_name;
+-----+-----+-----+
| s_id | s_name | avg_score |
+-----+-----+-----+
| 01   | 赵雷   | 88.67   |
| 02   | 钱电   | 70.00   |
| 03   | 孙风   | 82.00   |
| 04   | 李云   | 40.00   |
| 05   | 周梅   | 81.50   |
| 06   | 吴兰   | 49.50   |
| 07   | 郑竹   | 91.50   |
+-----+-----+-----+
7 rows in set (0.00 sec)
```

2、我们对上面的结果进行排序

!!! MySQL5 中是没有 rank 函数的，需要自己实现排序功能

```
1  -- MySQL5.7 中 没有rank函数，所以通过自连接实现
2
3  select
4      t1.s_id
5      ,t1.s_name
6      ,t1.avg_score
7      ,(select count(distinct t2.avg_score)
8         from (select
9                 sc.s_id
10                ,s.s_name
11                ,round(avg(sc.s_score),2) avg_score
12                from Score sc
13                join Student s
14                on sc.s_id=s.s_id
15                group by sc.s_id,s.s_name)t2    -- 临时表t2也是上面的结果
16         where t2.avg_score >= t1.avg_score
17         ) rank
18
19  from (select
```

```

20     sc.s_id
21     ,s.s_name
22     ,round(avg(sc.s_score),2) avg_score
23 from Score sc
24 join Student s
25 on sc.s_id=s.s_id
26 group by sc.s_id,s.s_name)t1    -- 临时表t1就是上面的结果
27 order by t1.avg_score desc;

```

```

+-----+-----+-----+-----+
| s_id | s_name | avg_score | rank |
+-----+-----+-----+-----+
| 07   | 郑竹   | 91.50    | 1    |
| 01   | 赵雷   | 88.67    | 2    |
| 03   | 孙风   | 82.00    | 3    |
| 05   | 周梅   | 81.50    | 4    |
| 02   | 钱电   | 70.00    | 5    |
| 06   | 吴兰   | 49.50    | 6    |
| 04   | 李云   | 40.00    | 7    |
+-----+-----+-----+-----+
7 rows in set (0.00 sec)

```

1.25.2.2 参考方法

```

1  select
2      a.s_id -- 学号
3      ,@i:=@i+1 as '不保留空缺排名'    -- 直接i的自加
4      ,@k:=(case when @avg_score=a.avg_s then @k else @i end) as '保留空缺排名'
5      ,@avg_score:=avg_s as '平均分'    -- 表a中的值
6
7  from (select
8          s_id
9          ,round(avg(s_score), 2) as avg_s
10         from Score
11         group by s_id
12         order by 2 desc)a    -- 表a: 平均成绩的排序和学号
13     ,(select @avg_score:=0, @i:=0, @k:=0)b    -- 表b: 通过变量设置初始值

```



```
mysql> select
-> a.s_id
-> ,@i:=@i+1 as '不保留空缺排名'
-> ,@k:=(case when @avg_score=a.avg_s then @k else @i end) as '保留空缺排名'
-> ,@avg_score:=avg_s as '平均分'
->
-> from (select
-> s_id
-> ,round(avg(s_score), 2) as avg_s
-> from Score
-> group by s_id
-> order by 2 desc)a
-> ,(select @avg_score:=0, @i:=0, @k:=0)b;
```

s_id	不保留空缺排名	保留空缺排名	平均分
07	1	1	91.50
01	2	2	88.67
03	3	3	82.00
05	4	4	81.50
02	5	5	70.00
06	6	6	49.50
04	7	7	40.00

7 rows in set (0.01 sec)

1.25.3 实现 rank 函数

```
1 select
2     s.s_name -- 姓名
3     ,s.s_score -- 成绩
4     ,(select count(distinct t2.s_score)
5     from Score t2
6     where t2.s_score >= t1.s_score) rank -- 在t2分数大的情况下，统计t2的去重
7     个数
8 from Score t1
9 order by t1.s_score desc; -- 分数降序排列
```

举例子来说明这个脚本：

姓名	成绩
----	----

张三	89
----	----

李四	90
----	----

王五	78
----	----

小明	98
----	----

小红	60
----	----

1. 当 t1.s_score=89，满足 t2.s_score >= t1.s_score 的有 **98, 90 和 89**，此时 count(distinct t2.s_score) 的个数就是 3

2. 当 $t1.s_score=90$, 满足 $t2.s_score \geq t1.s_score$ 的有 **98 和 90**, 此时 $\text{count}(\text{distinct } t2.s_score)$ 的个数就是 2
3. 当 $t1.s_score=78$, 满足 $t2.s_score \geq t1.s_score$ 的有 **98、90、89 和 78**, 此时 $\text{count}(\text{distinct } t2.s_score)$ 的个数就是 4
4. 当 $t1.s_score=98$, 满足 $t2.s_score \geq t1.s_score$ 的只有 **98**, 此时 $\text{count}(\text{distinct } t2.s_score)$ 的个数就是 1
5. 当 $t1.s_score=60$, 满足 $t2.s_score \geq t1.s_score$ 的有 **89、90、78、98、60**, 此时 $\text{count}(\text{distinct } t2.s_score)$ 的个数就是 5

通过上面的步骤, 我们发现: **t1 中每个分数对应的个数就是它的排名**

1.26 题目 25

1.26.1 题目需求

查询各科成绩前三名的记录

1.26.2 SQL 实现

1.26.2.1 自己的方法

1、首先我们找出语文的前 3 名

```
1 select s.s_id, s.s_score, c.c_name
2 from Score s
3 join Course c on s.c_id = c.c_id
4 where c.c_name = '语文'
5 order by s.s_score desc -- 降序之后取出前3条记录
6 limit 3;
```

```
mysql> select s.s_id, s.s_score, c.c_name
-> from Score s
-> join Course c on s.c_id = c.c_id
-> where c.c_name = '语文'
[
-> order by s.s_score desc;
+-----+-----+-----+
| s_id | s_score | c_name |
+-----+-----+-----+
| 01   |      80 | 语文   |
| 03   |      80 | 语文   |
| 05   |      76 | 语文   |
| 02   |      70 | 语文   |
| 04   |      50 | 语文   |
| 06   |      43 | 语文   |
+-----+-----+-----+
6 rows in set (0.00 sec)
```

所有记录

```
mysql> select s.s_id, s.s_score, c.c_name
-> from Score s
-> join Course c on s.c_id = c.c_id
-> where c.c_name = '语文'
-> order by s.s_score desc -- 降序之后取出前3条记录
[
-> limit 3;
+-----+-----+-----+
| s_id | s_score | c_name |
+-----+-----+-----+
| 01   |      80 | 语文   |
| 03   |      80 | 语文   |
| 05   |      76 | 语文   |
+-----+-----+-----+
3 rows in set (0.00 sec)
```

取出前3条

2、通过同样的方法我们可以求出数学和英语的前 3 条记录，然后通过 union 进行联结，有待优化 ☒

```
1 -- 自己的脚本
2
3 (select s.s_id, s.s_score, c.c_name
4 from Score s
5 join Course c on s.c_id = c.c_id
6 where c.c_name = '语文'
7 order by s.s_score desc -- 降序之后取出前3条记录
```

```
8  limit 3)
9
10 union
11
12 (select s.s_id, s.s_score, c.c_name
13  from Score s
14  join Course c on s.c_id = c.c_id
15  where c.c_name = '数 学'
16  order by s.s_score desc
17  limit 3)
18
19 union
20
21 (select s.s_id, s.s_score, c.c_name
22  from Score s
23  join Course c on s.c_id = c.c_id
24  where c.c_name = '英 语'
25  order by s.s_score desc
26  limit 3)
```

```
mysql> (select s.s_id, s.s_score, c.c_name
-> from Score s
-> join Course c on s.c_id = c.c_id
-> where c.c_name = '语文'
-> order by s.s_score desc -- 降序之后取出前3条记录
-> limit 3)
-> union
-> (select s.s_id, s.s_score, c.c_name
-> from Score s
-> join Course c on s.c_id = c.c_id
-> where c.c_name = '数学'
-> order by s.s_score desc
-> limit 3)
-> union
-> (select s.s_id, s.s_score, c.c_name
-> from Score s
-> join Course c on s.c_id = c.c_id
-> where c.c_name = '英语'
-> order by s.s_score desc
-> limit 3);
+-----+-----+-----+
| s_id | s_score | c_name |
+-----+-----+-----+
| 01   |      80 | 语文   |
| 03   |      80 | 语文   |
| 05   |      76 | 语文   |
| 01   |      90 | 数学   |
| 07   |      89 | 数学   |
| 05   |      87 | 数学   |
| 01   |      96 | 英语   |
| 07   |      94 | 英语   |
| 03   |      85 | 英语   |
+-----+-----+-----+
9 rows in set (0.00 sec)
```

1.26.2.2 参考方法

通过 Score 表的自连接，表 a 中的值小于表 b 中的值，排序之后我们取前 3

```
1 select
2     a.s_id
3     ,a.c_id
4     ,a.s_score -- 取出a中的成绩
```

```

5 from Score a
6 join Score b
7 on a.c_id = b.c_id
8 and a.s_score <= b.s_score  -- 表b中的成绩大
9 group by 1,2,3
10 having count(b.s_id) = 3
11 order by 2, 3 desc;

```

我们通过语文这个科目来理解上面的代码：前3名是80，80，76

```

[mysql> select * from Score where c_id=01;
+-----+-----+-----+
| s_id | c_id | s_score |
+-----+-----+-----+
| 01   | 01   |      80 |
| 02   | 01   |      70 |
| 03   | 01   |      80 |
| 04   | 01   |      50 |
| 05   | 01   |      76 |
| 06   | 01   |      43 |
+-----+-----+-----+
6 rows in set (0.00 sec)

```

语文成绩

```

1 -- 语文
2 select
3     a.s_id
4     ,a.c_id
5     ,a.s_score  -- 3、此时a表的成绩就是我们找的
6 from Score a
7 join Score b
8 on a.c_id = b.c_id
9 and a.s_score <= b.s_score  -- 1、判断a的分数小于等于b的分数，要带上等号
10 and a.c_id="01"
11 group by 1,2
12 having count(b.s_id) <= 3  -- 2、b中的个数至少有3个，应对分数相同的情形
13 order by 3 desc;

```

```
mysql> select
-> a.s_id
-> ,a.c_id
-> ,a.s_score -- a表的成绩
-> from Score a
-> join Score b
-> on a.c_id = b.c_id
-> and a.s_score <= b.s_score
-> and a.c_id="01"
-> group by 1,2
-> having count(b.s_id) <= 3
[
-> order by 3 desc;
+-----+-----+-----+
| s_id | c_id | s_score |
+-----+-----+-----+
| 01   | 01   |      80 |
| 03   | 01   |      80 |
| 05   | 01   |      76 |
+-----+-----+-----+
3 rows in set (0.00 sec)
```

```
1 -- 语文
2
3 select
4     a.s_id
5     ,a.c_id
6     ,a.s_score -- a表的成绩
7 from Score a
8 join Score b
9 on a.c_id = b.c_id
10 and a.s_score <= b.s_score -- 1、判断a的分数小于等于b的分数，要带上等号
11 group by 1,2,3
12 having count(b.s_id) <= 3 -- 2、b中的个数至少有3个，应对分数相同的情形
13 order by 2, 3 desc; -- 课程(2)的升序，成绩()3的降序
```

```
mysql> select
-> a.s_id
-> ,a.c_id
-> ,a.s_score -- a表的成绩
-> from Score a
-> join Score b
-> on a.c_id = b.c_id
-> and a.s_score <= b.s_score
-> group by 1,2,3
-> having count(b.s_id) <= 3
-> order by 2, 3 desc; -- 课程 (2) 的升序, 成绩 (3) 的降序
+-----+-----+-----+
| s_id | c_id | s_score |
+-----+-----+-----+
| 01 | 01 | 80 |
| 03 | 01 | 80 |
| 05 | 01 | 76 |
| 01 | 02 | 90 |
| 07 | 02 | 89 |
| 05 | 02 | 87 |
| 01 | 03 | 96 |
| 07 | 03 | 94 |
| 03 | 03 | 85 |
+-----+-----+-----+
9 rows in set (0.00 sec)
```

1.27 题目 26

1.27.1 题目需求

查询每门课被选修的学生数

1.27.2 SQL 实现

```
1 select
2     c.c_id
3     ,c.c_name
4     ,count(s.s_id)
5 from Course c
6 join Score s
7 on c.c_id = s.c_id
8 group by c.c_id;
```



```
mysql> select
-> c.c_id
-> ,c.c_name
-> ,count(s.s_id)
-> from Course c
-> join Score s
-> on c.c_id = s.c_id
[
-> group by c.c_id;
+-----+-----+-----+
| c_id | c_name | count(s.s_id) |
+-----+-----+-----+
| 01   | 语文   | 6              |
| 02   | 数学   | 6              |
| 03   | 英语   | 6              |
+-----+-----+-----+
3 rows in set (0.00 sec)
```

1.28 题目 27

1.28.1 题目需求

查询出只有两门课程的全部学生的学号和姓名

1.28.2 SQL 实现

having 条件是分组之后在执行的，where 语句是分组前先执行的

```
1 select
2     s.s_id
3     ,s.s_name
4 from Student s
5 join Score sc
6 on s.s_id = sc.s_id
7 group by 1,2
8 having count(sc.c_id) = 2;
```

```
mysql> select
-> s.s_id
-> ,s.s_name
-> from Student s
-> join Score sc
-> on s.s_id = sc.s_id
-> group by 1,2
-> having count(sc.c_id) = 2;
```

```
+-----+-----+
```

```
| s_id | s_name |
```

```
+-----+-----+
```

```
| 05   | 周梅   |
```

```
| 06   | 吴兰   |
```

```
| 07   | 郑竹   |
```

```
+-----+-----+
```

```
3 rows in set (0.00 sec)
```

指定课程数是2

1.29 题目 28

1.29.1 题目需求

查询男女生人数

1.29.2 SQL 实现

先看看数据：男女人数都是 4

```
mysql> select * from Student;
+-----+-----+-----+-----+
| s_id | s_name | s_birth | s_sex |
+-----+-----+-----+-----+
| 01   | 赵雷   | 1990-01-01 | 男   |
| 02   | 钱电   | 1990-12-21 | 男   |
| 03   | 孙风   | 1990-05-20 | 男   |
| 04   | 李云   | 1990-08-06 | 男   |
| 05   | 周梅   | 1991-12-01 | 女   |
| 06   | 吴兰   | 1992-03-01 | 女   |
| 07   | 郑竹   | 1989-07-02 | 女   |
| 08   | 王菊   | 1990-01-20 | 女   |
+-----+-----+-----+-----+
8 rows in set (0.00 sec)
```

```
1  -- 自己的方法
2  select
3      count(case when s_sex='男' then 1 end) as '男'
4      ,count(case when s_sex='女' then 1 end) as '女'
5  from Student;
6
7  -- 参考方法
8  select
9      s_sex
10     ,count(s_sex) as `人数`
11  from Student
12  group by s_sex;
```

```
mysql> select
    -> count(case when s_sex='男' then 1 end) as '男'
    -> ,count(case when s_sex='女' then 1 end) as '女'
    -> from Student;
+-----+-----+
| 男   | 女   |
+-----+-----+
|    4 |    4 |
+-----+-----+
1 row in set (0.00 sec)
```

```
mysql> -- 参考方法
mysql> select
    -> s_sex
    -> ,count(s_sex) as `人数`
    -> from Student
[    -> group by s_sex;
+-----+-----+
| s_sex | 人数 |
+-----+-----+
| 女    |    4 |
| 男    |    4 |
+-----+-----+
2 rows in set (0.00 sec)
```

1.30 题目 29

1.30.1 题目需求

查询名字中含有风字的学生信息

1.30.2 SQL 实现

先看看哪些同学的名字中有风

```
mysql> select * from Student;
+-----+-----+-----+-----+
| s_id | s_name | s_birth | s_sex |
+-----+-----+-----+-----+
| 01   | 赵雷   | 1990-01-01 | 男   |
| 02   | 钱电   | 1990-12-21 | 男   |
| 03   | 孙风   | 1990-05-20 | 男   |
| 04   | 李云   | 1990-08-06 | 男   |
| 05   | 周梅   | 1991-12-01 | 女   |
| 06   | 吴兰   | 1992-03-01 | 女   |
| 07   | 郑竹   | 1989-07-02 | 女   |
| 08   | 王菊   | 1990-01-20 | 女   |
+-----+-----+-----+-----+
8 rows in set (0.00 sec)
```

- 1 -- 模糊匹配：我们在两边都加上了%，考虑的是姓或者名字含有风，虽然风姓很少见
- 2 `select * from Student where s_name like "%风%";`

```
mysql> select * from Student where s_name like "%风%";
+-----+-----+-----+-----+
| s_id | s_name | s_birth | s_sex |
+-----+-----+-----+-----+
| 03   | 孙风   | 1990-05-20 | 男   |
+-----+-----+-----+-----+
1 row in set (0.00 sec)
```

1.31 题目 30

1.31.1 题目需求

查询同名同性的学生名单，并统计同名人数

1.31.2 SQL 实现

- 1、先看看班级的学生信息

```
mysql> select * from Student;
+-----+-----+-----+-----+
| s_id | s_name | s_birth | s_sex |
+-----+-----+-----+-----+
| 01   | 赵雷   | 1990-01-01 | 男   |
| 02   | 钱电   | 1990-12-21 | 男   |
| 03   | 孙风   | 1990-05-20 | 男   |
| 04   | 李云   | 1990-08-06 | 男   |
| 05   | 周梅   | 1991-12-01 | 女   |
| 06   | 吴兰   | 1992-03-01 | 女   |
| 07   | 郑竹   | 1989-07-02 | 女   |
| 08   | 王菊   | 1990-01-20 | 女   |
+-----+-----+-----+-----+
8 rows in set (0.00 sec)
```

现有的数据中没有同名的学生，但是当班级人数增多的时候很有可能在班级上出现同名的学生

2、假设有同名同性的学生

```
1 select
2     a.s_name
3     ,a.s_sex
4     ,count(*)
5 from Student a -- 同一个表的自连接
6 join Student b
7 on a.s_id != b.s_id -- 连接的时候不能是同一个人：学号保证，每个人的学号是唯一的，其他字段都可能重复
8 and a.s_sex = b.s_sex -- 性别相同
9 and a.s_name = b.s_name -- 名字相同
10 group by 1,2;
```

```
mysql> select
-> a.s_name
-> ,a.s_sex
-> ,count(*)
-> from Student a -- 同一个表的自连接
-> join Student b
-> on a.s_id != b.s_id -- 连接的时候不能是同一个人：学号保证，每个人的学号是唯一的
-> and a.s_sex = b.s_sex -- 性别相同
-> and a.s_name = b.s_name -- 名字相同
-> group by 1,2;
Empty set (0.00 sec)
```

如果真的有满足要求的数据，会显示的

1.32 题目 31

1.32.1 题目需求

查询 1990 年出生的学生信息

1.32.2 SQL 实现

```
1 select *
2 from Student
3 where s_birth like '1990%';    -- 模糊匹配
```

```
mysql> select *
      -> from Student
[      -> where s_birth like '1990%';
+-----+-----+-----+-----+
| s_id | s_name | s_birth   | s_sex |
+-----+-----+-----+-----+
| 01   | 赵雷   | 1990-01-01 | 男    |
| 02   | 钱电   | 1990-12-21 | 男    |
| 03   | 孙风   | 1990-05-20 | 男    |
| 04   | 李云   | 1990-08-06 | 男    |
| 08   | 王菊   | 1990-01-20 | 女    |
+-----+-----+-----+-----+
5 rows in set (0.00 sec)
```

1.33 题目 32

1.33.1 题目需求

查询每门课程的平均成绩，结果按平均成绩降序排列；平均成绩相同时，按课程编号 c_id 升序排列

1.33.2 SQL 实现

```
1 -- 自己的方法
2 select
3     c_id
4     ,round(avg(s_score),2) avg_score
5 from Score
6 group by 1
```

```
7 order by 2 desc, c_id; -- 指定字段和排序方法
```

```
mysql> select
-> c_id
-> ,round(avg(s_score),2) avg_score
-> from Score
-> group by 1
-> order by 2 desc, c_id; -- 指定字段和排序方法
+-----+-----+
| c_id | avg_score |
+-----+-----+
| 02   | 74.50    |
| 03   | 73.50    |
| 01   | 66.50    |
+-----+-----+
3 rows in set (0.00 sec)
```

如果想带上课程的名称，需要和 Course 表进行联结

```
1 -- 自己的方法
2 select
3     c.c_id
4     ,c.c_name
5     ,round(avg(sc.s_score),2) avg_score
6 from Score sc
7 join Course c
8 on sc.c_id = c.c_id
9 group by 1,2
10 order by 3 desc, c.c_id; -- 指定字段和排序方法
```



```
mysql> select
  -> c.c_id
  -> ,c.c_name
  -> ,round(avg(sc.s_score),2) avg_score
  -> from Score sc
  -> join Course c
  -> on sc.c_id = c.c_id
  -> group by 1,2
[  -> order by 3 desc, c.c_id;  -- 指定字段和排序方法
+-----+-----+-----+
| c_id | c_name | avg_score |
+-----+-----+-----+
| 02   | 数学   | 74.50    |
| 03   | 英语   | 73.50    |
| 01   | 语文   | 66.50    |
+-----+-----+-----+
3 rows in set (0.01 sec)
```

1.34 题目 33

1.34.1 题目需求

查询平均成绩大于等于 85 的所有学生的学号、姓名和平均成绩

1.34.2 SQL 实现

```
1  -- 自己的方法
2  select
3      sc.s_id
4      ,s.s_name
5      ,round(avg(sc.s_score),2) avg_score
6  from Score sc
7  join Student s
8  on sc.s_id = s.s_id
9  group by sc.s_id,s.s_name
10 having avg_score >= 85;
```

```
mysql> select
-> sc.s_id
-> ,s.s_name
-> ,round(avg(sc.s_score),2) avg_score
-> from Score sc
-> join Student s
-> on sc.s_id = s.s_id
-> group by sc.s_id,s.s_name
-> having avg_score >= 85;
+-----+-----+-----+
| s_id | s_name | avg_score |
+-----+-----+-----+
| 01  | 赵雷   | 88.67    |
| 07  | 郑竹   | 91.50    |
+-----+-----+-----+
2 rows in set (0.00 sec)
```

1.35 题目 34

1.35.1 题目需求

查询课程名称为**数学**，且分数低于 60 的学生姓名和分数

1.35.2 SQL 实现

```
1 select
2     s.s_name
3     ,sc.s_score
4 from Score sc    -- 成绩表
5 join Student s  -- 学生信息表
6 on sc.s_id = s.s_id
7 join Course c   -- 课程表，指定数学
8 on sc.c_id = c.c_id
9 where c.c_name = '数学'
10 and sc.s_score < 60; -- 指定成绩
```

```
mysql> select
-> s.s_name
-> ,sc.s_score
-> from Score sc    -- 成绩表
-> join Student s   -- 学生信息表
-> on sc.s_id = s.s_id
-> join Course c    -- 课程表, 指定数学
-> on sc.c_id = c.c_id
-> where c.c_name = '数学'
[
-> and sc.s_score < 60;  -- 指定成绩
+-----+-----+
| s_name | s_score |
+-----+-----+
| 李云   |      40 |
+-----+-----+
1 row in set (0.00 sec)
```

看看真正的数据，的确只有一个人满足

```
mysql> select * from Score where c_id = 02;
```

s_id	c_id	s_score
01	02	90
02	02	60
03	02	81
04	02	40
05	02	87
07	02	89

6 rows in set (0.00 sec)

数学是02，只有一个人满足

1.36 题目 35

1.36.1 题目需求

查询所有学生的课程及分数情况

1.36.2 SQL 实现

```

1 select
2     s.s_id
3     ,s.s_name
4     ,sum(case c.c_name when '语文' then sc.s_score else 0 end) as '语文' -- 语
      文 分 数
5     ,sum(case c.c_name when '数学' then sc.s_score else 0 end) as '数学'
6     ,sum(case c.c_name when '英语' then sc.s_score else 0 end) as '英语'
7     ,sum(sc.s_score) as '总分' -- 每个人的总分
8 from Student s
9 left join Score sc
10 on s.s_id = sc.s_id
11 left join Course c
12 on sc.c_id = c.c_id
13 group by s.s_id, s.s_name; -- 学号和姓名的分组

```

```

mysql> select
-> s.s_id
-> ,s.s_name
-> ,sum(case c.c_name when '语文' then sc.s_score else 0 end) as '语文'
-> ,sum(case c.c_name when '数学' then sc.s_score else 0 end) as '数学'
-> ,sum(case c.c_name when '英语' then sc.s_score else 0 end) as '英语'
-> ,sum(sc.s_score) as '总分'
-> from Student s
-> left join Score sc
-> on s.s_id = sc.s_id
-> left join Course c
-> on sc.c_id = c.c_id
-> group by s.s_id, s.s_name;
+-----+-----+-----+-----+-----+-----+
| s_id | s_name | 语文 | 数学 | 英语 | 总分 |
+-----+-----+-----+-----+-----+-----+
| 01 | 赵雷 | 80 | 90 | 96 | 266 |
| 02 | 钱电 | 70 | 60 | 80 | 210 |
| 03 | 孙风 | 80 | 81 | 85 | 246 |
| 04 | 李云 | 50 | 40 | 30 | 120 |
| 05 | 周梅 | 76 | 87 | 0 | 163 |
| 06 | 吴兰 | 43 | 0 | 56 | 99 |
| 07 | 郑竹 | 0 | 89 | 94 | 183 |
| 08 | 王菊 | 0 | 0 | 0 | NULL |
+-----+-----+-----+-----+-----+-----+
8 rows in set (0.00 sec)

```

1.37 题目 36

1.37.1 题目需求

查询任何一门课程成绩在 70 分以上的姓名、课程名称和分数

1.37.2 SQL 实现

```
1 select
2     s.s_name
3     ,c.c_name
4     ,sc.s_score
5 from Score sc    -- 成绩表
6 join Student s  -- 学生信息表
7 on sc.s_id = s.s_id
8 join Course c   -- 课程表
9 on sc.c_id = c.c_id
10 where sc.s_score > 70
11 group by s.s_name, c.c_name, sc.s_score;
```

```
mysql> select
-> s.s_name
-> ,c.c_name
-> ,sc.s_score
-> from Score sc    -- 成绩表
-> join Student s  -- 学生信息表
-> on sc.s_id = s.s_id
-> join Course c   -- 课程表
-> on sc.c_id = c.c_id
-> where sc.s_score > 70
[ -> group by s.s_name, c.c_name, sc.s_score;
+-----+-----+-----+
| s_name | c_name | s_score |
+-----+-----+-----+
| 周梅   | 数学   |      87 |
| 周梅   | 语文   |      76 |
| 孙风   | 数学   |      81 |
| 孙风   | 英语   |      85 |
| 孙风   | 语文   |      80 |
| 赵雷   | 数学   |      90 |
| 赵雷   | 英语   |      96 |
| 赵雷   | 语文   |      80 |
| 郑竹   | 数学   |      89 |
| 郑竹   | 英语   |      94 |
| 钱电   | 英语   |      80 |
+-----+-----+-----+
11 rows in set (0.01 sec)
```

1.38 题目 37

1.38.1 题目需求

查询不及格的课程

1.38.2 SQL 实现

```
1 select
2     sc.c_id
3     ,c.c_name
4     ,sc.s_score
5 from Score sc
6 join Course c
7 on sc.c_id = c.c_id
8 where sc.s_score < 60;
```

```
mysql> select
-> sc.c_id
-> ,c.c_name
-> ,sc.s_score
-> from Score sc
-> join Course c
-> on sc.c_id = c.c_id
[ -> where sc.s_score < 60;
```

```
+-----+-----+-----+
| c_id | c_name | s_score |
+-----+-----+-----+
| 01   | 语文   | 50      |
| 02   | 数学   | 40      |
| 03   | 英语   | 30      |
| 01   | 语文   | 43      |
| 03   | 英语   | 56      |
+-----+-----+-----+
5 rows in set (0.00 sec)
```

1.39 题目 38

1.39.1 题目需求

查询课程编号为 01 且课程成绩大于等于 80 的学生的学号和姓名

1.39.2 SQL 实现

```
1 select
2     sc.s_id
3     ,s.s_name
4     ,sc.s_score
5 from Score sc    -- 成绩表
6 join Student s  -- 学生信息表
7 on sc.s_id = s.s_id
8 join Course c   -- 课程表
9 on sc.c_id = c.c_id
10 where c.c_id = 01
11 and sc.s_score >= 80;
```

```
mysql> select
-> sc.s_id
-> ,s.s_name
-> ,sc.s_score
-> from Score sc    -- 成绩表
-> join Student s  -- 学生信息表
-> on sc.s_id = s.s_id
-> join Course c   -- 课程表
-> on sc.c_id = c.c_id
-> where c.c_id = 01
[ -> and sc.s_score >= 80;
+-----+-----+-----+
| s_id | s_name | s_score |
+-----+-----+-----+
| 01   | 赵雷   |      80 |
| 03   | 孙风   |      80 |
+-----+-----+-----+
2 rows in set (0.00 sec)
```

1.40 题目 39

1.40.1 题目需求

每门课程的学生人数

1.40.2 SQL 实现

```
1 select
2     c_id
3     ,count(s_id)
4 from Score
5 group by c_id;
```

```
mysql> select
-> c_id
-> ,count(s_id)
-> from Score
[ -> group by c_id;
+-----+-----+
| c_id | count(s_id) |
+-----+-----+
| 01   |          6  |
| 02   |          6  |
| 03   |          6  |
+-----+-----+
3 rows in set (0.00 sec)
```

如果想连接到课程名称：

```
1 -- 报 错 !!!
2 select
3     sc.c_id
4     ,c.c_name
```



```

5      ,count(sc.s_id)
6  from Score sc
7  join Course c
8  group by sc.c_id;

```

ERROR 1055 (42000): Expression #2 of SELECT list is not in GROUP BY clause and contains nonaggregated column “test.c.c_name” which is not functionally dependent on columns in GROUP BY clause; this is incompatible with sql_mode=only_full_group_by

```

mysql> select
-> .c.c_id
-> .c.c_name
-> .count(sc.s_id)
-> from Score sc
-> join Course c
-> group by sc.c_id;
ERROR 1055 (42000): Expression #2 of SELECT list is not in GROUP BY clause and contains nonaggregated column 'test.c.c_name' which is not functionally dependent on columns in GROUP BY clause; this is incompatible with sql_mode=only_full_group_by

```

解决的方法是将我们之前的结果作为临时表和 Course 表来连接查询：

```

1  select
2      c.c_name  课程名称
3      ,c.c_id   课程编号
4      ,t.num    人数
5  from Course c
6  join(select
7      c_id
8      ,count(s_id) num
9      from Score
10     group by c_id)t  -- 上面结果的临时表
11 on c.c_id = t.c_id;

```

```

+-----+-----+-----+
| 课程名称 | 课程编号 | 人数 |
+-----+-----+-----+
| 语文     | 01       | 6    |
| 数学     | 02       | 6    |
| 英语     | 03       | 6    |
+-----+-----+-----+
3 rows in set (0.00 sec)

```

1.41 题目 40

1.41.1 题目需求

查询选修“张三”老师所授课程的学生中，成绩最高的学生信息及其成绩

1.41.2 SQL 实现

1、我们先找出张三老师教了哪些课程

```
1 select
2     c.c_id
3     ,c.c_name
4 from Course c
5 join Teacher t
6 on c.t_id = t.t_id
7 where t.t_name = '张三';
```

```
mysql> select
-> c.c_id
-> ,c.c_name
-> from Course c
-> join Teacher t
-> on c.t_id = t.t_id
[ -> where t.t_name = '张三';
+-----+-----+
| c_id | c_name |
+-----+-----+
| 02   | 数学   |
+-----+-----+
1 row in set (0.00 sec)
```

2、找出哪些人修了数学

```
1 select
2     sc.s_id
3     ,sc.s_score
4 from Score sc
5 left join Course c
6 on sc.c_id = c.c_id
7 left join Teacher t
8 on c.t_id = t.t_id
9 where t.t_name = '张三';
```

```
mysql> select
-> sc.s_id      3个表的连接，找出哪些人修了数学
-> ,sc.s_score
-> from Score sc
-> left join Course c
-> on sc.c_id = c.c_id
-> left join Teacher t
-> on c.t_id = t.t_id
-> where t.t_name = '张三';

+-----+-----+
| s_id | s_score |
+-----+-----+
| 01  |      90 |
| 02  |      60 |
| 03  |      81 |
| 04  |      40 |
| 05  |      87 |
| 07  |      89 |
+-----+-----+
6 rows in set (0.00 sec)
```

3、通过 max 函数找出成绩的最高分

```
1 select
2     max(sc.s_score)
3 from Score sc
4 left join Course c
5 on sc.c_id = c.c_id
6 left join Teacher t
7 on c.t_id = t.t_id
8 where t.t_name = '张三';
```

```
mysql> select
-> max(sc.s_score)
-> from Score sc
-> left join Course c
-> on sc.c_id = c.c_id
-> left join Teacher t
-> on c.t_id = t.t_id
[ -> where t.t_name = '张三';
+-----+
| max(sc.s_score) |
+-----+
|                90 |
+-----+
1 row in set (0.00 sec)
```

4、连接 Student 表，找出学生信息

```
1 select
2     s.*
3     ,sc.s_score
4     ,sc.c_id
5     ,c.c_name
6 from Student s
7 left join Score sc
8 on s.s_id = sc.s_id
9 left join Course c
10 on sc.c_id = c.c_id
11 where sc.s_score in (select max(sc.s_score)      -- 最大值90分的确定
12                      from Score sc
13                      left join Course c
14                      on sc.c_id = c.c_id
15                      left join Teacher t
16                      on c.t_id = t.t_id
17                      where t.t_name = '张三');
```

```
mysql> select
-> s.*
-> ,sc.s_score
-> ,sc.c_id
-> ,c.c_name
-> from Student s
-> left join Score sc
-> on s.s_id = sc.s_id
-> left join Course c
-> on sc.c_id = c.c_id
-> where sc.s_score in (select max(sc.s_score) -- 最大值90分的确定
-> from Score sc
-> left join Course c
-> on sc.c_id = c.c_id
-> left join Teacher t
-> on c.t_id = t.t_id
-> where t.t_name = '张三');
+-----+-----+-----+-----+-----+-----+
| s_id | s_name | s_birth | s_sex | s_score | c_id | c_name |
+-----+-----+-----+-----+-----+-----+
| 01 | 赵雷 | 1990-01-01 | 男 | 90 | 02 | 数学 |
+-----+-----+-----+-----+-----+
1 row in set (0.01 sec)
```

1.42 题目 41

1.42.1 题目需求

查询不同课程成绩相同的学生的学生编号、课程编号、学生成绩

1.42.2 SQL 实现

```
1 select
2     a.s_id
3     ,a.c_id
4     ,a.s_score
5 from Score a
6 join Score b
7 on a.c_id != b.c_id
8 and a.s_score = b.s_score
9 and a.s_id != b.s_id;
```

```
mysql> select
-> a.s_id
-> ,a.c_id
-> ,a.s_score
-> from Score a
-> join Score b
-> on a.c_id != b.c_id
-> and a.s_score = b.s_score
-> and a.s_id != b.s_id;
+-----+-----+-----+
| s_id | c_id | s_score |
+-----+-----+-----+
| 02   | 03   |      80 |
| 01   | 01   |      80 |
| 03   | 01   |      80 |
| 02   | 03   |      80 |
+-----+-----+-----+
4 rows in set (0.00 sec)
```

我们对学号还需要去重下：

```
1 select
2     distinct a.s_id
3     ,a.c_id
4     ,a.s_score
5 from Score a
6 join Score b
7 on a.c_id != b.c_id
8 and a.s_score = b.s_score
9 and a.s_id != b.s_id;
```

```
mysql> select
-> distinct a.s_id
-> ,a.c_id
-> ,a.s_score
-> from Score a
-> join Score b
-> on a.c_id != b.c_id
-> and a.s_score = b.s_score
[ -> and a.s_id != b.s_id;
+-----+-----+-----+
| s_id | c_id | s_score |
+-----+-----+-----+
| 02   | 03   |      80 |
| 01   | 01   |      80 |
| 03   | 01   |      80 |
+-----+-----+-----+
3 rows in set (0.00 sec)
```

再看看原始的数据中是否符合要求：

```
+-----+-----+-----+
| s_id | c_id | s_score |
+-----+-----+-----+
| 01   | 01   | 80      |
| 02   | 01   | 70      |
| 03   | 01   | 80      |
| 04   | 01   | 50      |
| 05   | 01   | 76      |
| 06   | 01   | 43      |
| 01   | 02   | 90      |
| 02   | 02   | 60      |
| 03   | 02   | 81      |
| 04   | 02   | 40      |
| 05   | 02   | 87      |
| 07   | 02   | 89      |
| 01   | 03   | 96      |
| 02   | 03   | 80      |
| 03   | 03   | 85      |
| 04   | 03   | 30      |
| 06   | 03   | 56      |
| 07   | 03   | 94      |
+-----+-----+-----+
18 rows in set (0.01 sec)
```

1.43 题目 42

1.43.1 题目需求

查询每门功成绩最好的前两名

1.43.2 SQL 实现

1.43.2.1 自己的方法

还需要好好优化的 ☹

```

1  -- 先找出语文的前2名同学
2
3  select
4      c.c_id
5      ,sc.s_id
6      ,sc.s_score
7  from Score sc
8  join Course c
9  on sc.c_id = c.c_id
10 where c.c_name = '语文'    -- 改成数学和英语即可求出相应的信息
11 order by sc.s_score desc
12 limit 2;
```

```

mysql> select
-> c.c_id
-> ,sc.s_id
-> ,sc.s_score
-> from Score sc
-> join Course c
-> on sc.c_id = c.c_id
-> where c.c_name = '语文'    -- 改成数学和英语即可求出相应的信息
-> order by sc.s_score desc
-> limit 2;
+-----+-----+-----+
| c_id | s_id | s_score |
+-----+-----+-----+
| 01   | 01   |      80 |
| 01   | 03   |      80 |
+-----+-----+-----+
2 rows in set (0.00 sec)
```

将 3 门学科的信息进行拼接即可求出答案：

```

1  -- 最终脚本
2
3  (select
4      c.c_id
5      ,sc.s_id
6      ,sc.s_score
7  from Score sc
8  join Course c
9  on sc.c_id = c.c_id
10 where c.c_name = '语文'
11 order by sc.s_score desc
12 limit 2)
```

```
13
14 union
15
16 (select
17     c.c_id
18     ,sc.s_id
19     ,sc.s_score
20 from Score sc
21 join Course c
22 on sc.c_id = c.c_id
23 where c.c_name = '数 学 '
24 order by sc.s_score desc
25 limit 2)
26
27 union
28
29 (select
30     c.c_id
31     ,sc.s_id
32     ,sc.s_score
33 from Score sc
34 join Course c
35 on sc.c_id = c.c_id
36 where c.c_name = '英 语 '
37 order by sc.s_score desc
38 limit 2)
```

```
+-----+-----+-----+
| c_id | s_id | s_score |
+-----+-----+-----+
| 01   | 01   | 80      |
| 01   | 03   | 80      |
| 02   | 01   | 90      |
| 02   | 07   | 89      |
| 03   | 01   | 96      |
| 03   | 07   | 94      |
+-----+-----+-----+
6 rows in set (0.00 sec)
```

1.43.2.2 参考方法（好方法）

如何解决前几名排序的问题 太牛了

```

1 select
2     a.c_id
3     ,a.s_id
4     ,a.s_score
5 from Score a
6 where (select count(1)    -- count(1) 类似 count(*)：统计表b中分数大的数量
7        from Score b
8        where b.c_id=a.c_id  -- 课程相同
9        and b.s_score >= a.s_score) <= 2    -- 前2名
10 order by a.c_id;
```

```

mysql> select
-> a.c_id
-> ,a.s_id
-> ,a.s_score
-> from Score a
-> where (select count(1) from Score b where b.c_id=a.c_id and b.s_score >= a.s_score) <= 2
-> order by a.c_id;
+-----+-----+-----+
| c_id | s_id | s_score |
+-----+-----+-----+
| 01   | 01   | 80      |
| 01   | 03   | 80      |
| 02   | 01   | 90      |
| 02   | 07   | 89      |
| 03   | 01   | 96      |
| 03   | 07   | 94      |
+-----+-----+-----+
6 rows in set (0.00 sec)
```

首先我们看看真实的数据，我们以 01 课程来进行解释上面的代码：

b.s_score	a.s_score	count(1)[满足b.s_score >= a.s_score]
80	80,80,76,70,50,43	6
70	70,50,43	3
80	80,80,76,70,50,43	6
50	50,43	2 (👉)
76	76,70,50,43	4
43	43	1 (👉)

符合要求count(1)<=2的只有两种情况

```
mysql> select * from Score group by c_id,s_id,s_score;
+-----+-----+-----+
| s_id | c_id | s_score |
+-----+-----+-----+
| 01   | 01   | 80      |
| 02   | 01   | 70      |
| 03   | 01   | 80      |
| 04   | 01   | 50      |
| 05   | 01   | 76      |
| 06   | 01   | 43      |
| 01   | 02   | 90      |
| 02   | 02   | 60      |
| 03   | 02   | 81      |
| 04   | 02   | 40      |
| 05   | 02   | 87      |
| 07   | 02   | 89      |
| 01   | 03   | 96      |
| 02   | 03   | 80      |
| 03   | 03   | 85      |
| 04   | 03   | 30      |
| 06   | 03   | 56      |
| 07   | 03   | 94      |
+-----+-----+-----+
18 rows in set (0.00 sec)
```

还需要好好理解下 ☹

1.44 题目 43

1.44.1 题目需求

统计每门课程的学生选修人数（超过 5 人的课程才统计）。要求输出课程号和选修人数，查询结果按人数降序排列，若人数相同，按课程号升序排列

1.44.2 SQL 实现

```
1 select
2     c_id
3     ,count(s_score) num
4 from Score
5 group by c_id
6 having num > 5
7 order by num desc, c_id;
```

```
mysql> select
-> c_id
-> ,count(s_score) num
-> from Score
-> group by c_id
-> having num > 5
[
-> order by num desc, c_id;
+-----+-----+
| c_id | num |
+-----+-----+
| 01   | 6   |
| 02   | 6   |
| 03   | 6   |
+-----+-----+
3 rows in set (0.00 sec)
```

人数相同，学号升序

1.45 题目 44

1.45.1 题目需求

检索至少选修两门课程的学生学号

1.45.2 SQL 实现

结果显示全部满足要求

```
1 select
2     s_id
3     ,count(*) num
4 from Score
5 group by s_id
6 having num >= 2;
```

```
mysql> select
      -> s_id
      -> ,count(*) num
      -> from Score
      -> group by s_id
[      -> having num >= 2;
+-----+-----+
| s_id | num |
+-----+-----+
| 01   | 3   |
| 02   | 3   |
| 03   | 3   |
| 04   | 3   |
| 05   | 2   |
| 06   | 2   |
| 07   | 2   |
+-----+-----+
7 rows in set (0.00 sec)
```

1.46 题目 45

1.46.1 题目需求

查询选修了全部课程的学生信息

1.46.2 SQL 实现

1.46.2.1 自己的方法

1、全部的课程数目 num

```
1 select count(*) from Course; -- 总共 3 门
```

```
mysql> select count(*) from Course;
+-----+
| count(*) |
+-----+
|          3 |
+-----+
1 row in set (0.00 sec)
```

2、从 Score 表分组统计每个人的课程数目，满足是 3 的学生信息

```
1 select
2     s_id
3     ,count(c_id) num    -- 课程数目
4 from Score
5 group by s_id
6 having num in (select count(*)
7                from Course); -- 满足全部课程
```

```
mysql> select
-> s_id
-> ,count(c_id) num    -- 课程数目
-> from Score
-> group by s_id
-> having num in (select count(*)
->                from Course); -- 满足全部课程
+-----+-----+
| s_id | num |
+-----+-----+
| 01   | 3   |
| 02   | 3   |
| 03   | 3   |
| 04   | 3   |
+-----+-----+
4 rows in set (0.00 sec)
```

3、我们找出上面结果中的学生信息即可

```
1 select
2     s.*
```

```

3      ,count(c_id) num    -- 课程数目
4  from Score sc
5  join Student s
6  on sc.s_id = s.s_id
7  group by s.s_id
8  having num in (select count(*)
9                  from Course); -- 满足全部课程

```

```

mysql> select
-> s.*
-> ,count(c_id) num    -- 课程数目
-> from Score sc
-> join Student s
-> on sc.s_id = s.s_id
-> group by s.s_id
-> having num in (select count(*)
->                  from Course); -- 满足全部课程

```

s_id	s_name	s_birth	s_sex	num
01	赵雷	1990-01-01	男	3
02	钱电	1990-12-21	男	3
03	孙风	1990-05-20	男	3
04	李云	1990-08-06	男	3

4 rows in set (0.00 sec)

1.46.2.2 参考方法

```

1  select *    -- 3、s_id对应的学生信息
2  from Student
3  where s_id in(select s_id -- 2、最大课程数对应的s_id
4                  from Score
5                  group by s_id
6                  having count(*)=(select count(*) from Course) -- 1、全部课程数
7                  )

```

1.47 题目 46

1.47.1 题目需求

查询各学生的年龄：按照出生日期来算，当前月日 < 出生年月的月日则，年龄减 1

参考：

https://blog.csdn.net/qq2712193/article/details/48766575?utm_medium=distribute.pc_aggpage_search_result.task-blog-2_all_sobaiduend~default-2-48766575.nonecase&utm_term=mysql%E6%AF%E8%BE%E4%B8

1.47.2 SQL 实现

1.47.2.1 自己的方法

```
1  -- 自己的方法
2
3  select *
4      ,case when dayofyear(now()) >= dayofyear(s_birth) then year(now()) - year(
         s_birth)
5          when dayofyear(now()) < dayofyear(s_birth) then year(now()) - year(
         s_birth) - 1
6      else 'other' end as 'age'
7  from Student;
```

```
+-----+-----+-----+-----+
| s_id | s_name | s_birth | s_sex | age |
+-----+-----+-----+-----+
| 01 | 赵雷 | 1990-01-01 | 男 | 30 |
| 02 | 钱电 | 1990-12-21 | 男 | 29 |
| 03 | 孙风 | 1990-05-20 | 男 | 30 |
| 04 | 李云 | 1990-08-06 | 男 | 30 |
| 05 | 周梅 | 1991-12-01 | 女 | 28 |
| 06 | 吴兰 | 1992-03-01 | 女 | 28 |
| 07 | 郑竹 | 1989-07-02 | 女 | 31 |
| 08 | 王菊 | 1990-01-20 | 女 | 30 |
+-----+-----+-----+-----+
8 rows in set (0.00 sec)
```

1.47.2.2 参考方法

```
1 select
2     s_name
3     ,s_birth
4     ,date_format(now(), '%Y') - date_format(s_birth, '%Y') - (case when
        date_format(now(), '%m%d') > date_format(s_birth, '%m%d') then 0 else 1
        end) as age    -- 当前日期大，说明已经过生了，年龄正常；反之说明今年还没有
        到年龄-1
5 from Student;
```

```
mysql> select
--> s_name
--> s_birth
--> date_format(now(), '%Y') - date_format(s_birth, '%Y') - (case when date_format(now(), '%m%d') > date_format(s_birth, '%m%d') then 0 else 1 end) as age
-- 当前日期大, 说明已经过生了, 年龄正常; 反之说明今年还没有到年龄-1
--> from Student;
+-----+
| s_name | s_birth | age |
+-----+
| 赵雷   | 1990-01-01 | 30 |
| 钱电   | 1990-12-21 | 29 |
| 孙凤   | 1990-05-20 | 30 |
| 李云   | 1990-08-06 | 30 |
| 周梅   | 1991-12-01 | 28 |
| 吴兰   | 1992-03-01 | 28 |
| 郑竹   | 1989-07-02 | 31 |
| 王菊   | 1990-01-20 | 30 |
+-----+
8 rows in set (0.00 sec)
```

1.47.3 如何返回年份/日期

通过date_format函数能够指定返回的数据

```
1 -- 两个方法
2
3 select year(now());
4 select date_format(now(), '%Y');
```

```
[mysql> select year(now());
+-----+
| year(now()) |
+-----+
|          2020 |
+-----+
```

```
[mysql> select date_format(now(), '%Y');
+-----+
| date_format(now(), '%Y') |
+-----+
| 2020                      |
+-----+
1 row in set (0.00 sec)
```

返回具体的日期:

```
mysql> select date_format(s_birth, '%m%d') from Student;
+-----+
| date_format(s_birth, '%m%d') |
+-----+
| 0101                          |
| 1221                          |
| 0520                          |
| 0806                          |
| 1201                          |
| 0301                          |
| 0702                          |
| 0120                          |
+-----+
8 rows in set (0.00 sec)
```

1.48 题目 47

1.48.1 题目需求

查询本周过生日的学生

```
1 select week(now()); -- 47
```

1. DAYOFWEEK(date)

返回日期 date 的星期索引 (1= 星期天, 2= 星期一,7= 星期六), 符合国内标准

2. WEEKDAY(date)

返回 date 的星期索引 (0= 星期一, 1= 星期二,6= 星期天), 国外标准

1.48.2 SQL 实现

1.48.2.1 自己的方法

```
1 select *
2 from Student
3 where week(s_birth) = week(now());
```

```
mysql> select *
      -> from Student
      -> where week(s_birth) = week(now());
Empty set (0.00 sec)
```

数据中没有11月份的同学，显示空值

```
mysql> select * from Student;
+-----+-----+-----+-----+
| s_id | s_name | s_birth | s_sex |
+-----+-----+-----+-----+
| 01 | 赵雷 | 1990-01-01 | 男 |
| 02 | 钱电 | 1990-12-21 | 男 |
| 03 | 孙风 | 1990-05-20 | 男 |
| 04 | 李云 | 1990-08-06 | 男 |
| 05 | 周梅 | 1991-12-01 | 女 |
| 06 | 吴兰 | 1992-03-01 | 女 |
| 07 | 郑竹 | 1989-07-02 | 女 |
| 08 | 王菊 | 1990-01-20 | 女 |
+-----+-----+-----+-----+
8 rows in set (0.00 sec)
```

1.48.2.2 参考方法

```
1 select * from Student where week(date_format(now(), '%Y%m%d')) = week(s_birth);
   -- 方式1
2
3 select * from student where yearweek(s_birth) = yearweek(date_format(now(), '%Y%
   m%d')); -- 方式2
```

1.49 题目 48

1.49.1 题目需求

查询下周过生日的学生

1.49.2 SQL 实现

```
1 -- 自己的方法
2 select *
3 from Student
4 where week(s_birth) = week(now()) + 1; -- 往前推1周
```

```

5
6 -- 参考方法
7 select * from Student where week(date_format(now(), '%Y%m%d')) + 1 = week(s_birth
);

```

```

mysql> select *
      -> from Student
      -> where week(s_birth) = week(now()) + 1 -- 往前推1周
+-----+-----+-----+-----+
| s_id | s_name | s_birth   | s_sex |
+-----+-----+-----+-----+
| 05   | 周梅   | 1991-12-01 | 女    |
+-----+-----+-----+-----+
1 row in set (0.01 sec)

```

1.49.3 边界问题

如果现在刚好的是今年的最后一个周，那么下周就是明年的第一个周，我们如何解决这个问题呢?? 改进后的脚本：

```

1 -- 自己的方法
2
3 select * from Student
4 where mod(week(now()), 52) + 1 = week(s_birth);

```

当现在刚好是第 52 周，那么 mod 函数的结果是 0，则说明出生的月份刚好是明年的第一周

1.50 题目 49

1.50.1 题目需求

查询本月过生的同学

```

mysql> select month(now());
+-----+
| month(now()) |
+-----+
|          11 |
+-----+
1 row in set (0.00 sec)

```

1.50.2 SQL 实现

```

1  -- 自己的方法
2  select *
3  from Student
4  where month(s_birth) = month(now());
5
6  -- 参考方法
7  select * from Student where month(date_format(now(), '%Y%m%d')) = month(s_birth
    );

```

```

mysql> select *
      -> from Student
[      -> where month(s_birth) = month(now());
Empty set (0.00 sec)

```

返回的是空值，是因为数据本身就没有在 11 月份出生的同学

```

mysql> select * from Student;
+-----+-----+-----+-----+
| s_id | s_name | s_birth   | s_sex |
+-----+-----+-----+-----+
| 01   | 赵雷   | 1990-01-01 | 男    |
| 02   | 钱电   | 1990-12-21 | 男    |
| 03   | 孙风   | 1990-05-20 | 男    |
| 04   | 李云   | 1990-08-06 | 男    |
| 05   | 周梅   | 1991-12-01 | 女    |
| 06   | 吴兰   | 1992-03-01 | 女    |
| 07   | 郑竹   | 1989-07-02 | 女    |
| 08   | 王菊   | 1990-01-20 | 女    |
+-----+-----+-----+-----+
8 rows in set (0.00 sec)

```

本身数据就没有11月份的

1.51 题目 50

1.51.1 题目需求

查询下月过生的同学

1.51.2 SQL 实现

```

1  -- 自己的方法
2  select * from Student
3  where month(s_birth) = month(now()) + 1;  -- 推迟一个月
4
5  -- 参考方法
6  select * from Student
7  where month(date_format(now(), '%Y%m%d')) + 1 = month(s_birth);

```

```

mysql> select * from Student
[    -> where month(s_birth) = month(now()) + 1;  -- 推迟一个月
+-----+-----+-----+-----+
| s_id | s_name | s_birth   | s_sex |
+-----+-----+-----+-----+
| 02   | 钱电   | 1990-12-21 | 男    |
| 05   | 周梅   | 1991-12-01 | 女    |
+-----+-----+-----+-----+
2 rows in set (0.00 sec)

```

1.51.3 边界问题

假设现在是 12 月份，那么下个月就是明年的 1 月份，我们如何解决??? 将上面的代码进行改进：

```

1  select * from Student
2  where mod(month(now()),12) + 1 = month(s_birth);

```

```

mysql> select * from Student
[    -> where mod(month(now()),12) + 1 = month(s_birth);
+-----+-----+-----+-----+ 当前月份与12的余数
| s_id | s_name | s_birth   | s_sex |
+-----+-----+-----+-----+
| 02   | 钱电   | 1990-12-21 | 男    |
| 05   | 周梅   | 1991-12-01 | 女    |
+-----+-----+-----+-----+
2 rows in set (0.00 sec)

```

如果现在是 12 月份，则 mod 函数的结果是 0，说明生日刚好是 1 月份

本文作者简介

Peter，硕士毕业僧一枚，从电子专业自学 Python 入门数据行业，擅长数据分析及可视化。喜欢数据，坚持跑步，热爱阅读，乐观生活。个人格言：不浮于世，不负于己

个人站点：www.renpeter.cn，欢迎常来小屋逛逛

2 延伸阅读

2.1 我是谁

大家好，我是 Lemon，公众号「Python 数据之道」号主。

公众号「Python 数据之道」秉承“让数据更有价值”的理念，主要分享数据相关的内容，包括数据分析，挖掘，可视化，机器学习，深度学习等，希望能给大家分享有价值的内容。

若对我写的内容有兴趣，欢迎大家通过以下途径来关注。

2.1.1 微信公众号



Figure 2.1: Python 数据之道

「Python 数据之道」是我分享关于 Python 及数据分析相关内容的主阵地。

2.1.2 个人网站

网址：<http://liyangbit.com>

Lemon 的个人网站中，包含更多的文章，并且在不断的进行更新。目前，网站中涉及了 Python 相关一系列内容，包括 Python 基础、Python 数据科学、项目实战等内容，欢迎访问。

2.2 推荐内容

2.2.1 《Python 知识手册》

「Python 数据之道」整理并出品了《Python 知识手册》，大家可以在公众号「Python 数据之道」后台回复数字「600」来获取高清 PDF 版。



Figure 2.2: Python 知识手册

2.2.2 《Python 时间使用指南》

大家可以在公众号「柠檬数据」后台回复「time」来获取《Python 时间使用指南》高清 PDF 版。



Figure 2.3: 柠檬数据

以下是部分内容的思维导图：

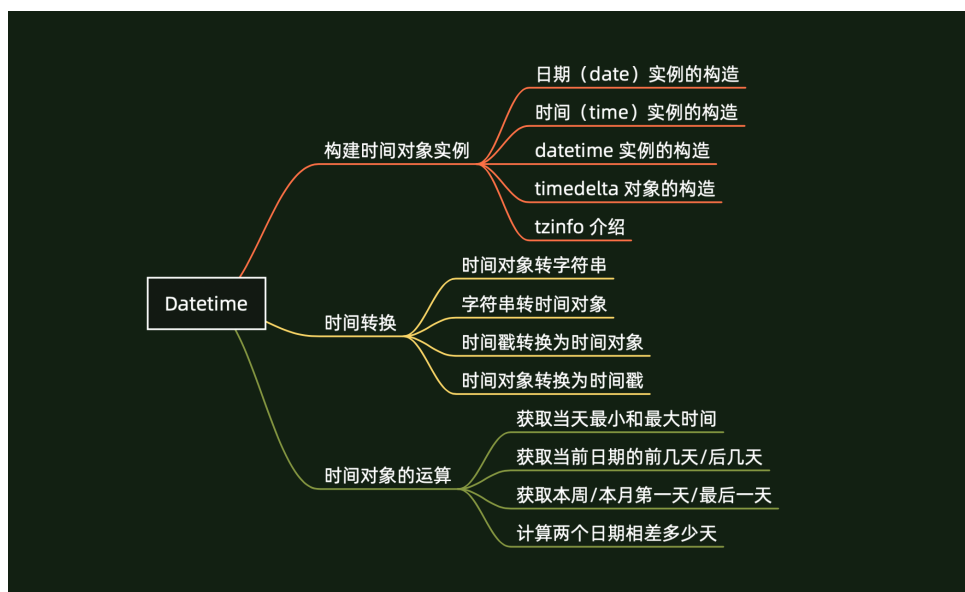


Figure 2.4: datetime 模块

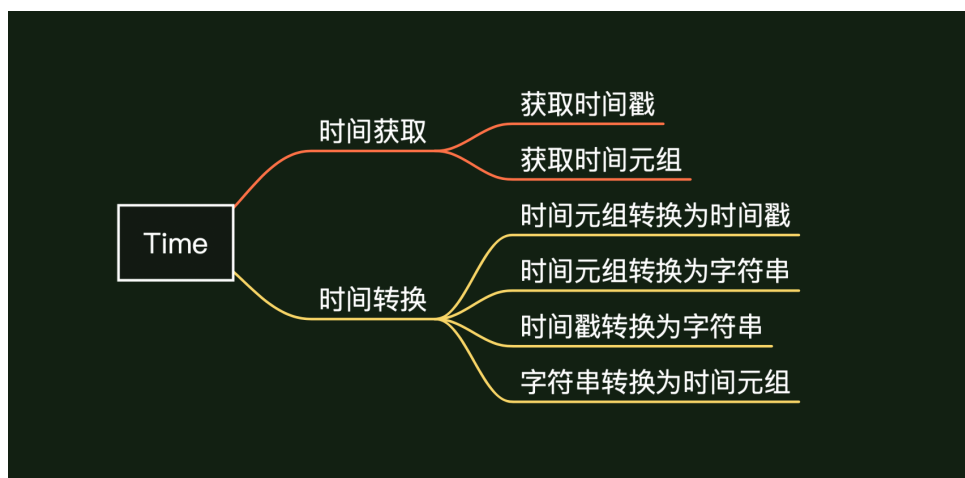


Figure 2.5: time 模块

2.2.3 精选书单

Lemon 是一个爱好读书的人，平时喜欢在公众号「价值前瞻」上写一些读书笔记内容，目前发布的有《如何阅读一本书》、《巴菲特之道》、《金字塔原理》、高瓴张磊的《价值》、《投资最重要的事》等书籍的笔记内容和思维导图。

大家可以在公众号「价值前瞻」后台回复「书单」来获取精选书单和思维导图。



Figure 2.6: 价值前瞻

部分书籍内容如下：



Figure 2.7: 《价值》

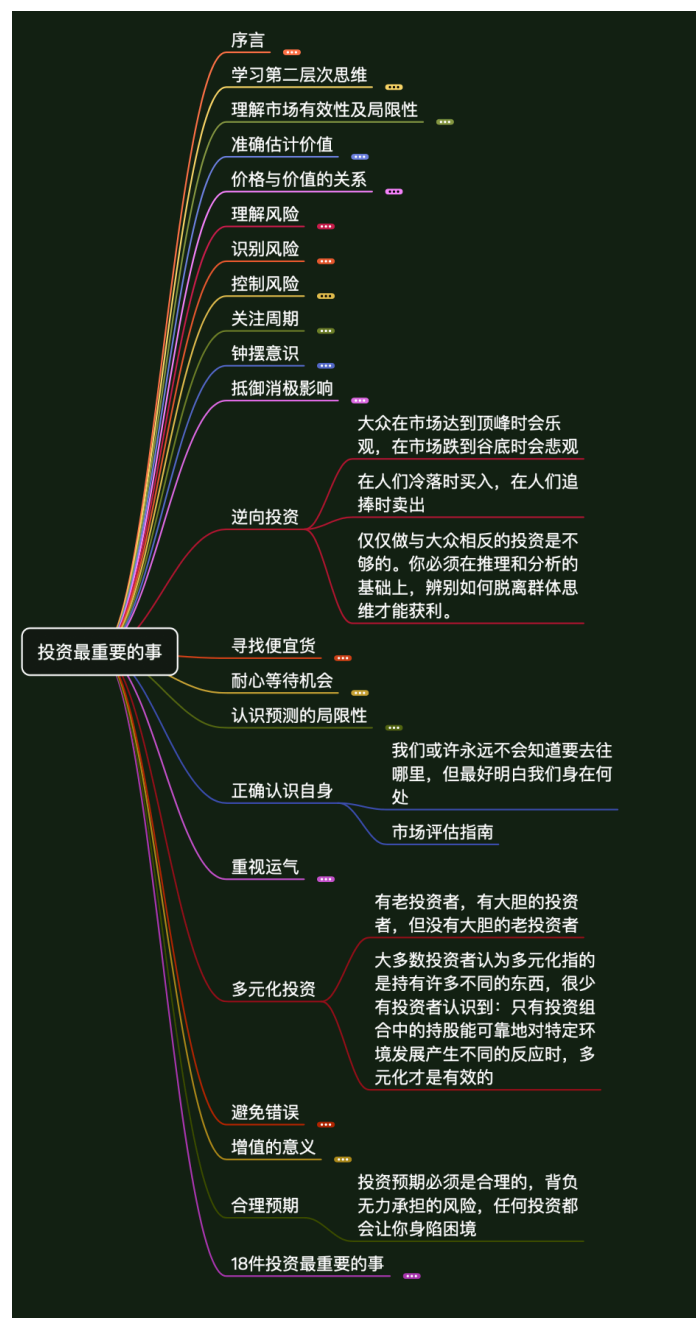


Figure 2.8: 《投资最重要的事》

2.2.4 精选文章

- 用 Python 可视化神器 Plotly 动态演示全球疫情变化趋势
- 用 Plotly 动态柱状图来演示全球疫情变化趋势

- [超火动态排序疫情变化图](#), 这次我们用 Plotly 来绘制
- [用 Python 动态曲线图来对全球疫情进行演示](#)
- [升级版](#), 用 Python 来进行多条曲线动态演示全球疫情变化
- [深度好文](#), Matplotlib 可视化最有价值的 50 个图表 (附完整 Python 源代码)
- [用 Python 读取巴菲特近期持仓数据](#)
- [推荐一个牛逼的生物信息 Python 库 - Dash Bio](#)
- [轻松用 Seaborn 进行数据可视化](#)
- [用 Python 快速分析和预测股票价格](#)
- [干货推荐: 轻松玩转 Bokeh 可视化 \(项目实战经验分享\)](#)
- [巧用 Matplotlib 动画](#), 让你的 Python 可视化大放异彩